

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCMUTE

ĐỒ ÁN MÔN HỌC 2

**THIẾT KẾ IP SPI GIAO TIẾP APB VÀ KIỂM
THỬ VỚI UVM**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: **VÕ QUANG HUY**

MSSV: 22119082

HUỖNH MINH QUỶ

MSSV: 22119125

TP. HỒ CHÍ MINH – 12/2025

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG

ĐỒ ÁN MÔN HỌC 2

**THIẾT KẾ IP SPI GIAO TIẾP APB VÀ KIỂM
THỬ VỚI UVM**

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: **VÕ QUANG HUY**
MSSV: 22119082
HUỲNH MINH QUÝ
MSSV: 22119125

Hướng dẫn: **ThS. LÊ MINH**

TP. HỒ CHÍ MINH – 12/2025

BẢN NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

LỜI CẢM ƠN

Nhóm xin chân thành cảm ơn ThS. Lê Minh – giảng viên hướng dẫn đã tận tình chỉ bảo, hỗ trợ và định hướng cho nhóm trong suốt quá trình thực hiện đề tài. Sự hướng dẫn chuyên môn sâu sắc và những góp ý quý báu từ thầy là yếu tố quan trọng giúp nhóm hoàn thiện đề tài này.

Nhóm cũng xin gửi lời cảm ơn đến các thầy cô trong khoa đã tạo điều kiện học tập và nghiên cứu thuận lợi, cùng với bạn bè, gia đình đã luôn động viên, giúp đỡ nhóm trong suốt quá trình thực hiện đề tài.

Mặc dù đã cố gắng nhưng không tránh khỏi những thiếu sót, nhóm rất mong nhận được sự góp ý của thầy cô để hoàn thiện hơn trong những lần nghiên cứu sau.

Trân trọng cảm ơn!

TÓM TẮT

Đề tài “Thiết kế IP SPI giao tiếp APB và kiểm thử với UVM” được thực hiện nhằm đáp ứng nhu cầu ngày càng cao về các khối IP giao tiếp tốc độ cao, dễ tích hợp trong hệ thống SoC, đồng thời đảm bảo tính tin cậy và khả năng tái sử dụng trong thiết kế vi mạch. Vấn đề đặt ra là cần xây dựng một IP SPI có thể tương thích với giao thức APB, đảm bảo hoạt động ổn định, và đồng thời có cơ chế kiểm thử tự động để phát hiện lỗi sớm trong giai đoạn thiết kế. Khác với nhiều nghiên cứu trước chủ yếu sử dụng testbench truyền thống, đề tài áp dụng môi trường UVM và phát triển SPI VIP để tự động hóa, tăng độ tin cậy và khả năng tái sử dụng trong kiểm thử. Đề tài không chỉ tập trung vào thiết kế IP mà còn nhấn mạnh vào việc kết hợp chặt chẽ với môi trường UVM giúp nâng cao hiệu quả kiểm thử, khả năng mở rộng và tái sử dụng cho các dự án sau. Kết quả đạt được là một khối IP SPI – APB hoạt động chính xác theo đặc tả, được kiểm chứng qua nhiều trường hợp kiểm thử, chứng minh được độ tin cậy và sẵn sàng tích hợp vào các hệ thống SoC thực tế.

MỤC LỤC

CHƯƠNG 1	GIỚI THIỆU	1
1.1	GIỚI THIỆU	1
1.2	MỤC TIÊU ĐỀ TÀI	1
1.3	PHẠM VI GIỚI HẠN ĐỀ TÀI	2
1.4	PHƯƠNG PHÁP NGHIÊN CỨU	2
1.5	ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU	2
1.6	BỐ CỤC QUYỀN BÁO CÁO	3
CHƯƠNG 2	CƠ SỞ LÝ THUYẾT	4
2.1	TỔNG QUAN VỀ AMBA VÀ GIAO THỨC APB	4
2.1.1	Tổng quan về AMBA	4
2.1.2	Giới thiệu về giao thức APB3	5
2.1.3	Hệ thống tín hiệu của giao thức APB3	5
2.1.4	Quá trình ghi của giao thức APB3	6
2.1.5	Quá trình đọc của giao thức APB3	8
2.1.6	Phản hồi lỗi của giao thức APB3	10
2.1.7	Trạng thái hoạt động của giao thức APB3	11
2.2	TỔNG QUAN VỀ CHUẨN GIAO TIẾP SPI	12
2.2.1	Giới thiệu về chuẩn giao tiếp SPI	12
2.2.2	Các tín hiệu của SPI	12
2.2.3	Nguyên lý hoạt động của SPI	13
2.3	PHƯƠNG PHÁP XÁC MINH UVM	14
2.3.1	Tổng quan về UVM	14
2.3.2	Cấu trúc môi trường UVM	16
2.3.2.1	Lớp testbench	16
2.3.2.2	Lớp uvm_test	17
2.3.2.3	Lớp uvm_env	17
2.3.2.4	Lớp uvm_agent	17
2.3.2.5	Lớp uvm_sequencer	18

2.3.2.6	Lớp uvm_driver.....	18
2.3.2.7	Lớp uvm_monitor	18
2.3.2.8	Lớp uvm_scoreboard.....	19
2.3.3	Các cơ chế trong môi trường UVM.....	19
2.3.3.1	Cơ chế phân pha (UVM Phasing)	19
2.3.3.2	Cơ chế bắt tay giữa sequencer và driver	20
2.3.3.3	Cơ chế mô hình hoá giao dịch (Transaction-Level Modeling - TLM)	21
2.3.3.4	Cơ chế đăng kí Factory	22
2.3.3.5	Cơ chế uvm_config_db	22
2.3.3.6	Cơ chế Register Abstraction Layer (RAL)	23
CHƯƠNG 3	THIẾT KẾ HỆ THỐNG	25
3.1	YÊU CẦU HỆ THỐNG.....	25
3.1.1	Yêu cầu thiết kế IP SPI master giao tiếp qua APB3	25
3.1.2	Yêu cầu thiết kế môi trường kiểm thử UVM	25
3.2	THIẾT KẾ HỆ THỐNG PHẦN CỨNG	26
3.2.1	Chức năng hệ thống.....	26
3.2.2	Sơ đồ khối hệ thống.....	27
3.2.3	Thiết kế khối APB Slave	28
3.2.3.1	Sơ đồ khối	28
3.2.3.2	Cách hoạt động.....	30
3.2.4	Bộ thanh ghi	30
3.2.4.1	Sơ đồ khối	30
3.2.4.2	Hệ thống thanh ghi	33
3.2.4.3	Phản hồi lỗi.....	37
3.2.5	Thiết kế khối TX FIFO và RX FIFO.....	37
3.2.5.1	Sơ đồ khối	37
3.2.5.2	Cách hoạt động.....	39
3.2.6	Thiết kế khối SPI Master.....	40
3.2.6.1	Sơ đồ khối	40

3.2.6.2	Quy trình hoạt động	43
3.3	THIẾT KẾ MÔI TRƯỜNG KIỂM THỬ UVM.....	45
3.4	THIẾT KẾ KỊCH BẢN KIỂM THỬ	46
3.4.1	Kiểm tra hoạt động của các thanh ghi	46
3.4.2	Kiểm tra tương thích với nhiều tần số APB	47
3.4.3	Kiểm tra hoạt động SPI	47
3.4.4	Kiểm tra hoạt động của cấu hình ngắt.....	47
3.4.5	Kiểm tra hoạt động cấu hình động	47
3.4.6	Kiểm tra hoạt động cấu hình lỗi	48
3.4.7	Kiểm tra việc ghi đè cấu hình khi SPI đang hoạt động	48
CHƯƠNG 4	KẾT QUẢ.....	49
4.1	KẾT QUẢ THỰC HIỆN THIẾT KẾ PHẦN CỨNG	49
4.2	KẾT QUẢ THỰC HIỆN THIẾT KẾ MÔI TRƯỜNG KIỂM THỬ UVM	49
4.3	HOẠT ĐỘNG CỦA HỆ THỐNG	51
4.3.1	Kiểm tra hệ thống thanh ghi	51
4.3.2	Kiểm tra hoạt động truyền nhận dữ liệu.....	53
4.3.2.1	Chế độ truyền không liên tục	53
4.3.2.2	Chế độ truyền liên tục	55
4.3.3	Kiểm tra hoạt động chống ghi đè LCR và DLR khi SPI đang hoạt động	57
4.3.4	Đánh giá độ bao phủ chức năng	58
CHƯƠNG 5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	60
5.1	KẾT LUẬN	60
5.1.1	Kết quả đạt được so với mục tiêu đặt ra.....	60
5.1.2	Ưu nhược điểm của hệ thống đã thiết kế.....	61
5.2	HƯỚNG PHÁT TRIỂN.....	61
PHỤ LỤC		62
TÀI LIỆU THAM KHẢO		63

DANH MỤC HÌNH

Hình 2.1. Sơ đồ khối giao tiếp giữa APB và AHB	5
Hình 2.2. Quá trình ghi không có trạng thái chờ.....	7
Hình 2.3. Quá trình ghi có trạng thái chờ.....	8
Hình 2.4. Quá trình đọc không có trạng thái chờ	9
Hình 2.5. Quá trình đọc có trạng thái chờ	9
Hình 2.6. Phản hồi lỗi trong quá trình ghi.....	10
Hình 2.7. Phản hồi lỗi trong quá trình đọc	10
Hình 2.8. Sơ đồ trạng thái hoạt động của APB3	11
Hình 2.9. Giao diện SPI với Master và Slave	12
Hình 2.10. Dạng sóng truyền dữ liệu và lấy mẫu chế độ khác nhau của SPI	13
Hình 2.11. Hệ thống phân cấp trong môi trường UVM.....	14
Hình 2.12. Sơ đồ tổ chức UVM testbench tiêu chuẩn.....	16
Hình 2.13. Cấu trúc phân cấp cơ chế pha của UVM.....	19
Hình 2.14. Cơ chế bắt tay giữa sequencer và driver	20
Hình 2.15. Cơ chế mô hình hoá giao dịch (TLM).....	21
Hình 2.16. Cơ chế uvm_config_db	22
Hình 2.17. Cơ chế Register Abstraction Layer (RAL)	23
Hình 3.1. Sơ đồ khối tổng quát của IP SPI giao tiếp APB.....	27
Hình 3.2. Sơ đồ khối tổng quát khối APB Slave.....	28
Hình 3.3. Sơ đồ khối tổng quát bộ thanh ghi	31
Hình 3.4. Sơ đồ khối tổng quát bộ FIFO.....	38
Hình 3.5. Cấu trúc bộ FIFO.....	39
Hình 3.6. Sơ đồ khối tổng quát khối SPI Master	40
Hình 3.7. Sơ đồ kết nối các khối chính bên trong SPI Master	42
Hình 3.8. Quy trình hoạt động bên trong khối SPI Master	43
Hình 3.9. Cấu trúc môi trường kiểm thử UVM.....	45
Hình 4.1. Sơ đồ nguyên lý IP SPI giao tiếp APB.....	49
Hình 4.2. Kết quả cấu trúc phân cấp	50
Hình 4.3. Kết quả kiểm tra giá trị mặc định của các thanh ghi	52
Hình 4.4. Kết quả kiểm tra giá trị trong quá trình đọc/ghi của các thanh ghi	53

Hình 4.5. Kết quả quá trình truyền không liên tục.....	54
Hình 4.6. Giai đoạn cấu hình.....	54
Hình 4.7. Quá trình truyền nhận dữ liệu	55
Hình 4.8. Kết quả quá trình truyền liên tục	56
Hình 4.9. Cấu hình của chế độ truyền liên tục	56
Hình 4.10. Truyền nhận của chế độ truyền liên tục	57
Hình 4.11. Kiểm tra chống ghi đè LCR	57
Hình 4.12. Kiểm tra chống ghi đè DLR	58
Hình 4.13. Mô hình đánh giá độ bao phủ chức năng	59

DANH MỤC BẢNG

Bảng 2.1. Mô tả tín hiệu của giao thức APB3.....	5
Bảng 3.1. Danh sách các chân tín hiệu của khối APB Slave	29
Bảng 3.2. Danh sách các chân tín hiệu của bộ thanh ghi	31
Bảng 3.3. Tóm tắt hệ thống thanh ghi	33
Bảng 3.4. Mô tả thanh ghi điều khiển đường truyền - LCR.....	34
Bảng 3.5. Mô tả thanh ghi chốt hệ số chia - DLR.....	35
Bảng 3.6. Mô tả thanh ghi cho phép ngắt - IER.....	35
Bảng 3.7. Mô tả thanh ghi trạng thái FIFO - FSR.....	36
Bảng 3.8. Mô tả thanh ghi đệm truyền - TBR.....	36
Bảng 3.9. Mô tả thanh ghi đệm nhận - RBR	37
Bảng 3.10. Danh sách các chân tín hiệu của bộ FIFO.....	38
Bảng 3.11. Danh sách chân tín hiệu của khối SPI Master.....	40

CÁC TỪ VIẾT TẮT

AHB	Advanced High-performance Bus
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
ARM	Advanced RISC Machine
ASIC	Application-Specific Integrated Circuit
AXI	Advanced eXtensible Interface
CPU	Central Processing Unit
CPHA	Clock Phase
CPOL	Clock Polarity
DMA	Direct Memory Access
DLR	Divisor Latch Register
DUT	Device Under Test
FIFO	First-In First-Out
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
FSR	FIFO Status Register
GPIO	General Purpose Input/Output
I2C	Inter-Integrated Circuit
IER	Interrupt Enable Register
IP	Intellectual Property
LCR	Line Control Register
MCU	Microcontroller Unit
MISO	Master In Slave Out

MOSI	Master Out Slave In
MSB	Most Significant Bit
OOP	Object-Oriented Programming
RAL	Register Abstraction Layer
RAM	Random Access Memory
RAZ	Read As Zero
RBR	Receive Buffer Register
RO	Read Only
RTL	Register Transfer Level
R/W	Read / Write
SCLK	Serial Clock
SoC	System on Chip
SPI	Serial Peripheral Interface
SS/CS	Slave Select / Chip Select
TBR	Transmit Buffer Register
TLM	Transaction-Level Modeling
UART	Universal Asynchronous Receiver/Transmitter
UVM	Universal Verification Methodology
VIF	Virtual Interface
VIP	Verification IP
WI	Write Ignore
WO	Write Only

CHƯƠNG 1

GIỚI THIỆU

1.1 GIỚI THIỆU

Trong lĩnh vực thiết kế vi mạch số, xu hướng phát triển các hệ thống SoC (System on Chip) đang ngày càng được mở rộng. Đây là dạng mạch tích hợp các thành phần bao gồm bộ vi xử lý trung tâm (CPU), bộ nhớ và các hệ thống ngoại vi. Sự phát triển ngày càng nhanh của SoC kéo theo nhu cầu tích hợp nhiều chuẩn ngoại vi để đảm bảo khả năng tối ưu và tích hợp với nhiều hệ thống. Trong số đó, SPI (Serial Peripheral Interface) là một trong những chuẩn giao tiếp nối tiếp đồng bộ phổ biến, có cấu trúc đơn giản, tốc độ cao và được sử dụng rộng rãi trong các hệ thống nhúng.

Song song với đó, APB (Advanced Peripheral Bus), thuộc họ AMBA được phát triển bởi ARM, là một giao thức chuyên dùng trong việc giao tiếp với các ngoại vi trong hệ thống. Đây là giao thức không cần tốc độ xử lý cao, băng thông thấp và phù hợp trong việc giao tiếp với các ngoại vi như I2C, SPI, UART hay GPIO.

Đề tài “Thiết kế IP SPI giao tiếp APB và kiểm thử với UVM” được lựa chọn nhằm nghiên cứu, thiết kế và sử dụng phương pháp xác minh UVM (Universal Verification Methodology) trong việc xác minh IP SPI.

1.2 MỤC TIÊU ĐỀ TÀI

Đề tài “Thiết kế IP SPI giao tiếp APB và kiểm thử với UVM” hướng tới việc thiết kế khối IP là cầu nối giữa bus APB và chuẩn truyền thông SPI với các mục tiêu sau:

- Thiết kế và mô tả RTL IP SPI với ngôn ngữ Verilog ở chế độ Master với các chức năng truyền nhận dữ liệu theo chuẩn SPI, giao tiếp với APB. hỗ trợ cấu hình CPOL (Clock Polarity) và CPHA (Clock Phase).
- Phát triển môi trường kiểm thử theo chuẩn UVM.
- Mô phỏng, kiểm tra và đánh giá hoạt động của IP SPI.

1.3 PHẠM VI GIỚI HẠN ĐỀ TÀI

Việc nghiên cứu, phát triển IP SPI chỉ dừng lại ở mức thiết kế RTL và mô phỏng xác minh, không đi sâu vào các bước tổng hợp (synthesis) hay thiết kế vật lý (layout). Trong đó, IP được thiết kế hoạt động với chế độ SPI master, không hỗ trợ chế độ SPI slave, và giao tiếp với giao thức APB3 hay AMBA 3 APB. Đề tài chỉ dừng lại ở mức thiết kế RTL và mô phỏng, việc kiểm tra và xác minh chỉ thực hiện trên môi trường UVM, không triển khai lên các thiết bị ASIC hay FPGA.

1.4 PHƯƠNG PHÁP NGHIÊN CỨU

Phương pháp tổng hợp tài liệu: Tổng hợp, nghiên cứu các tài liệu kỹ thuật về chuẩn giao tiếp SPI, giao thức APB và môi trường xác minh UVM.

Phương pháp thiết kế phần cứng: Đặc tả chi tiết hệ thống, thiết kế sơ đồ khối và mô tả RTL hệ thống.

Phương pháp xác minh: Phát triển môi trường kiểm thử UVM, triển khai VIP (Verification IP) và tiến hành xác minh hệ thống qua các trường hợp nằm trong kế hoạch cần kiểm tra.

1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu: Chuẩn giao tiếp SPI (chế độ Master), giao thức APB và phương pháp xác minh bằng UVM.

Phạm vi nghiên cứu: Giới hạn trong việc thiết kế và kiểm thử một khối IP SPI có giao diện bus APB3 ở mức RTL. Việc đánh giá chỉ dừng lại ở mức mô phỏng, chưa triển khai trên phần cứng thực tế và chưa mở rộng sang các chuẩn giao tiếp khác.

1.6 BỐ CỤC QUYỀN BÁO CÁO

Chương 1. GIỚI THIỆU.

Trình bày tổng quan về đề tài nghiên cứu, bao gồm mục tiêu thực hiện, giới hạn đề tài, phương pháp nghiên cứu, đối tượng và phạm vi nghiên cứu.

Chương 2. CƠ SỞ LÝ THUYẾT.

Giới thiệu các kiến thức nền tảng liên quan đến môi trường kiểm thử theo chuẩn UVM, chuẩn giao tiếp SPI và giao thức APB, làm cơ sở cho việc thiết kế và triển khai đề tài.

Chương 3. THIẾT KẾ HỆ THỐNG.

Trình bày chi tiết về sơ đồ khối, kiến trúc và chức năng của IP SPI, cũng như quy trình xây dựng môi trường kiểm thử UVM.

Chương 4. KẾT QUẢ.

Đưa ra các kết quả đạt được sau khi hoàn thành hệ thống, bao gồm kết quả mô phỏng, kiểm thử chức năng, phân tích và so sánh với mục tiêu ban đầu đã đề ra.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.

Tổng kết lại những kết quả chính của đề tài, đánh giá ưu điểm và hạn chế, đồng thời đề xuất các hướng phát triển trong tương lai để cải thiện và mở rộng khả năng ứng dụng của hệ thống.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 TỔNG QUAN VỀ AMBA VÀ GIAO THỨC APB

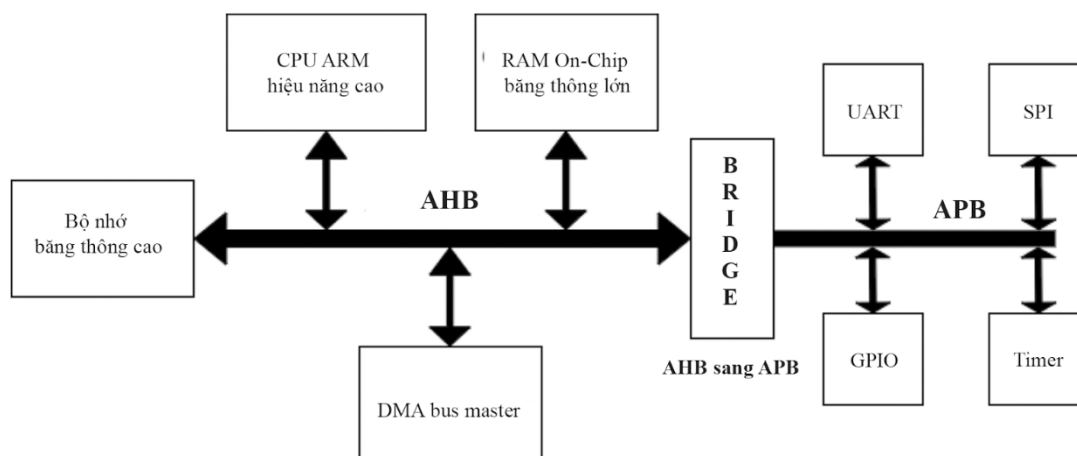
2.1.1 Tổng quan về AMBA

AMBA (Advanced Microcontroller Bus Architecture) là kiến trúc bus do ARM phát triển, được sử dụng phổ biến trong hệ thống SoC và vi điều khiển. Đây là một tiêu chuẩn mở, miễn phí, đóng vai trò là chuẩn giao tiếp nội bộ, kết nối bộ xử lý trung tâm, bộ nhớ và các khối ngoại vi hiệu quả. Nhờ sự phổ biến rộng rãi, AMBA có cho mình một hệ sinh thái đối tác rộng rãi, đảm bảo khả năng tương thích và mở rộng giữa các thành phần IP. Từ đó, AMBA giúp đơn giản hóa quá trình thiết kế phần cứng và tăng khả năng tái sử dụng các IP trong nhiều dự án khác nhau.

Trong kiến trúc AMBA, các chuẩn bus được chia theo hiệu năng và nhu cầu sử dụng. Trong đó, phổ biến nhất là AXI, AHB và APB. AXI (Advanced eXtensible Interface) là chuẩn bus có hiệu năng cao nhất, hỗ trợ giao tiếp dữ liệu song song và pipeline được dùng làm giao thức tiêu chuẩn cho SoC tầm trung và cao, kết nối CPU với các mô-đun như bộ tăng tốc (accelerator), đồ họa, hay mạng (Ethernet và WiFi). AHB (Advanced High-performance Bus) có cấu trúc đơn giản hơn AXI, vẫn đảm bảo hiệu năng tương đối cao và thường được sử dụng kết nối các mô-đun trong các MCU nhỏ như Flash, RAM và DMA. Trong khi đó, APB (Advanced Peripheral Bus) là dạng bus có cấu trúc đơn giản nhất, phù hợp với các mô-đun có tần số thấp như GPIO, Timer, I2C, SPI, UART. Nhờ sự phân chia hợp lý này, AMBA đáp ứng tốt các yêu cầu về hiệu năng, chi phí và độ phức tạp trong các hệ thống SoC hiện đại.

2.1.2 Giới thiệu về giao thức APB3

APB (Advanced Peripheral Bus) là kiến trúc bus cơ bản nhất của AMBA, được thiết kế kết nối với các thiết bị ngoại vi tốc độ thấp, tiêu thụ năng lượng thấp và không yêu cầu băng thông cao.



Hình 2.1. Sơ đồ khối giao tiếp giữa APB và AHB

APB hoạt động dựa trên mô hình Master-Slave. Trong đó chỉ có một APB master, thường là cầu nối APB với các bus có tốc độ cao hơn như AHB, AXI, điều khiển giao dịch. Các khối ngoại vi kết nối với APB bus đóng vai trò là Slave hoạt động và giao tiếp theo các tín hiệu điều khiển của Master.

2.1.3 Hệ thống tín hiệu của giao thức APB3

Tín hiệu của giao thức APB là các tín hiệu cho phép APB master giao tiếp với IP thông các cách thao tác điều khiển các thanh ghi, đọc các tín hiệu trạng thái phản hồi. Các tín hiệu thuộc APB3 bao gồm:

Bảng 2.1. Mô tả tín hiệu của giao thức APB3

Tín hiệu	Nguồn tín hiệu	Mô tả
PCLK	Bộ tạo xung hệ thống	Tín hiệu xung clock sử dụng cạnh lên điều khiển hoạt động các giao tiếp APB
PRESETn	Bus hệ thống	Tín hiệu reset mức thấp đặt lại hệ thống APB về chế độ mặc định
PADDR	APB Master	Thông tin địa chỉ thanh ghi cho quá trình đọc và

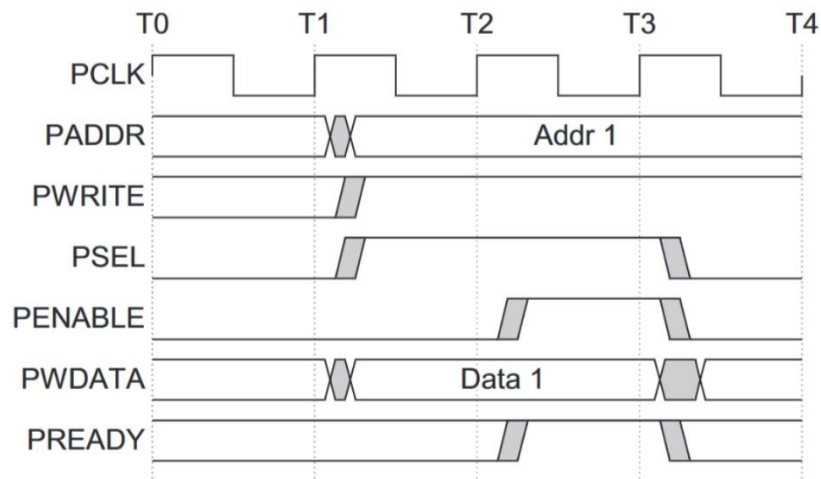
		ghi dữ liệu
PSELx	APB Master	Tín hiệu chọn Slave. Master tích cực tín hiệu PSEL của Slave tương ứng để bắt đầu giao dịch
PENABLE	APB Master	Tín hiệu cho biết thời điểm bắt đầu giai đoạn truy cập (ACCESS phase)
PWRITE	APB Master	Tín hiệu cho biết loại giao dịch với mức thấp cho giao dịch đọc và cao cho ghi
PWDATA	APB Master	Dữ liệu được ghi vào thanh ghi của Slave trong giao dịch ghi
PRDATA	APB Slave	Dữ liệu được đọc ra từ thanh ghi của Slave trong giao dịch đọc
PREADY	APB Slave	Tín hiệu sẵn sàng của Slave yêu cầu trạng thái chờ khi không thể hoàn tất quá trình giao dịch
PSLVERR	APB Slave	Tín hiệu phản hồi lỗi của Slave, mức cao cho biết giao dịch thất bại.

Các tín hiệu trong giao thức APB3 được tổ chức rõ ràng để hỗ trợ việc truyền dữ liệu đơn giản và hiệu quả. Trong đó, các tín hiệu hệ thống như PCLK và PRESETn đảm bảo đồng bộ và khởi tạo hệ thống; Tín hiệu điều khiển PSEL, PENABLE, PADDR, PWRITE xác định thời điểm, địa chỉ, loại giao dịch và được hoàn tất thông qua các tín hiệu dữ liệu PWDATA, PRDATA. Song song với đó, các tín hiệu phản hồi PREADY và PSLVERR cho biết tình trạng của Slave và giao dịch.

2.1.4 Quá trình ghi của giao thức APB3

Việc ghi dữ liệu của APB3 được chia thành 2 kiểu là: Quá trình ghi không có trạng thái chờ và quá trình ghi có trạng thái chờ.

Quá trình ghi không có trạng thái chờ được mô tả theo hình sau:



Hình 2.2. Quá trình ghi không có trạng thái chờ

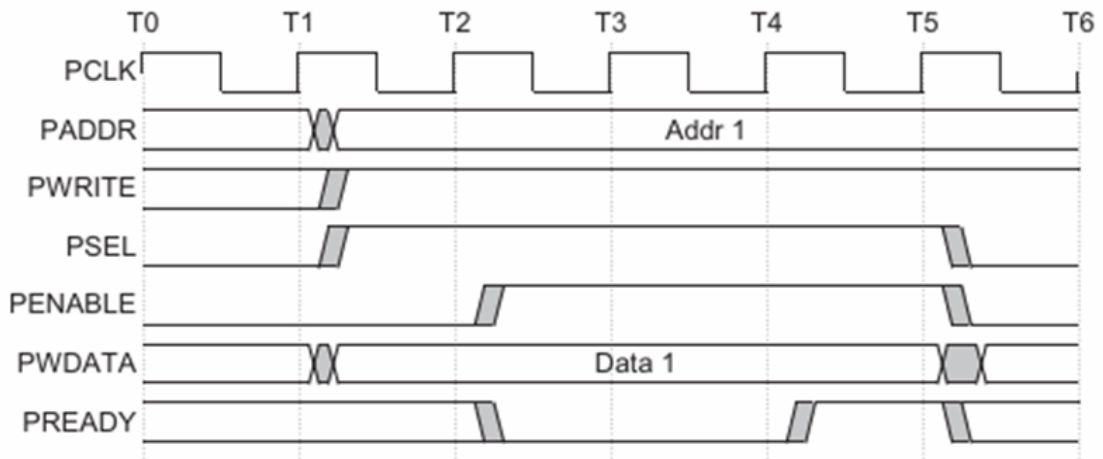
Ở chu kỳ T0 - T1, bus APB ở trạng thái nhàn rỗi, tất cả các tín hiệu điều khiển như PSEL, PENABLE, và PREADY đều không hoạt động.

Tại cạnh lên của PCLK ở T1, APB master bắt đầu giai đoạn thiết lập (Setup Phase) bằng cách tích cực mức cao PSEL và PWRITE cho giao dịch ghi, đồng thời cung cấp địa chỉ và dữ liệu cần ghi qua PADDR và PWDATA.

Tại T2, APB master kéo PENABLE lên mức cao để chuyển sang giai đoạn truy cập (Access Phase) và báo cho APB slave rằng dữ liệu và địa chỉ trên bus đã ổn định và có thể được chấp nhận. Vì đây là trường hợp không có trạng thái chờ (no wait state), tín hiệu PREADY từ Slave được duy trì ở mức cao ngay trong chu kỳ này, cho phép giao dịch được hoàn tất trong cùng một xung clock. Trong suốt quá trình này, các giá trị trên PADDR và PWDATA được giữ ổn định để đảm bảo dữ liệu truyền đi chính xác.

Tại T3, Master kết thúc quá trình truyền bằng cách đưa tín hiệu PSEL và PENABLE về mức thấp, báo hiệu kết thúc một giao dịch ghi hoàn chỉnh. Tín hiệu PSEL có thể được giữ ở mức cao nếu Master dự định thực hiện một giao dịch tiếp theo mà không cần hủy chọn Slave.

Quá trình ghi có trạng thái chờ được mô tả theo hình sau:



Hình 2.3. Quá trình ghi có trạng thái chờ

Với giao dịch ghi có trạng thái chờ, từ T0 - T2, Master vẫn điều khiển Slave như ghi không có trạng thái chờ đến hết giai đoạn thiết lập (Setup Phase).

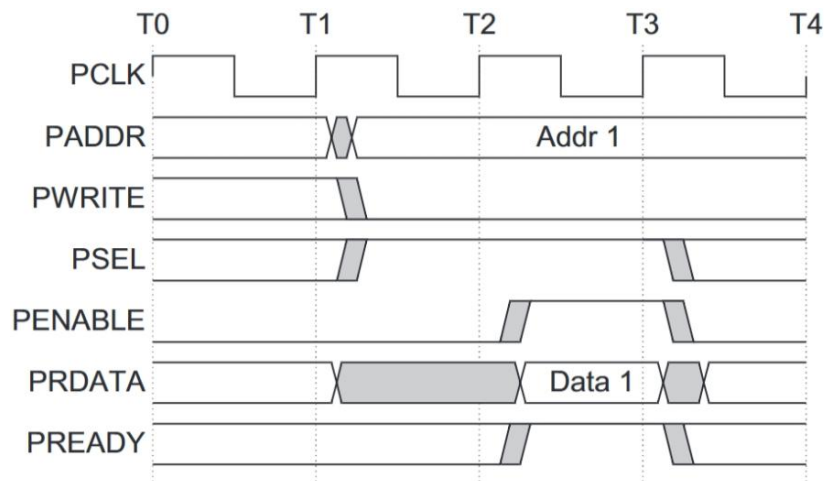
Tại cạnh lên của chu kỳ T2, APB master kéo PENABLE lên mức cao để bắt đầu giai đoạn truy cập (Access Phase). Nhưng từ T2 đến T4, Slave kéo PREADY xuống mức thấp để báo rằng Slave chưa sẵn sàng vào giai đoạn truy cập. Lúc này, Master sẽ chờ đến khi PREADY được bật lên ở T4 để chính thức vào giai đoạn truy cập và tiến hành ghi dữ liệu vào thanh ghi.

Sau khi kết thúc giai đoạn truy cập ở T5, Master chuyển Slave sang trạng thái nhàn rỗi. Trong quá trình này, PSEL có thể vẫn giữ ở mức cao nếu Master dự định tiếp tục một giao dịch mới với cùng Slave mà không cần hủy chọn.

2.1.5 Quá trình đọc của giao thức APB3

Giống như quá trình ghi, quá trình đọc dữ liệu của APB3 cũng được chia thành hai kiểu là: đọc không có trạng thái chờ và đọc có trạng thái chờ.

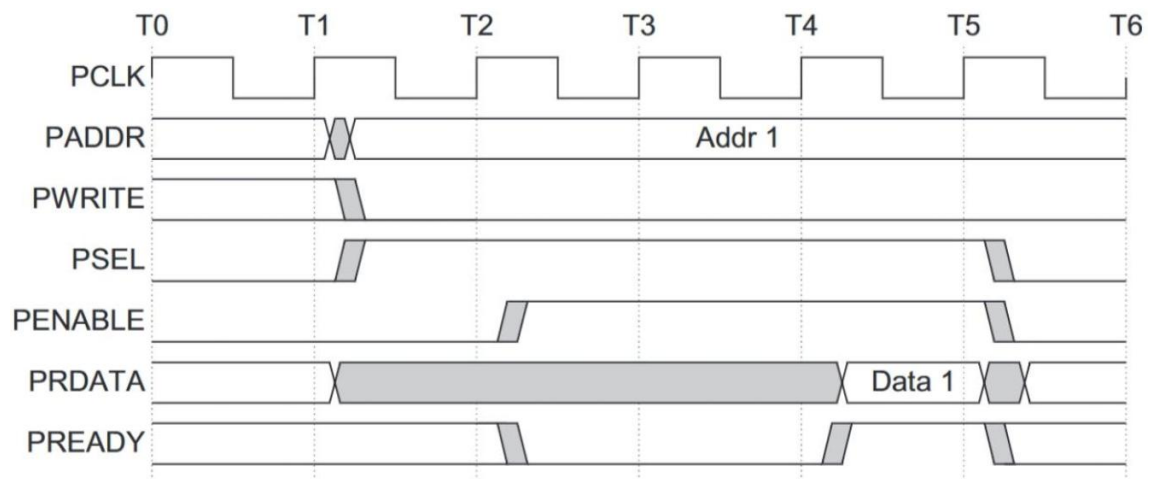
Quá trình đọc không có trạng thái chờ được mô tả theo hình sau:



Hình 2.4. Quá trình đọc không có trạng thái chờ

Tương tự như quá trình ghi, Master cũng tích cực PSEL để chọn Slave và bắt đầu giai đoạn thiết lập (Setup phase) ở T1 nhưng thay vì ở mức cao, PWRITE được hạ xuống mức thấp để thông báo kiểu giao dịch đọc cho Slave. Sau đó tại T2, Master bật PENABLE để vào giai đoạn truy cập, lúc này dữ liệu của thanh ghi có địa chỉ được Master cung cấp suốt quá trình sẽ được đưa ra qua PRDATA.

Quá trình đọc có trạng thái chờ được mô tả theo hình sau:



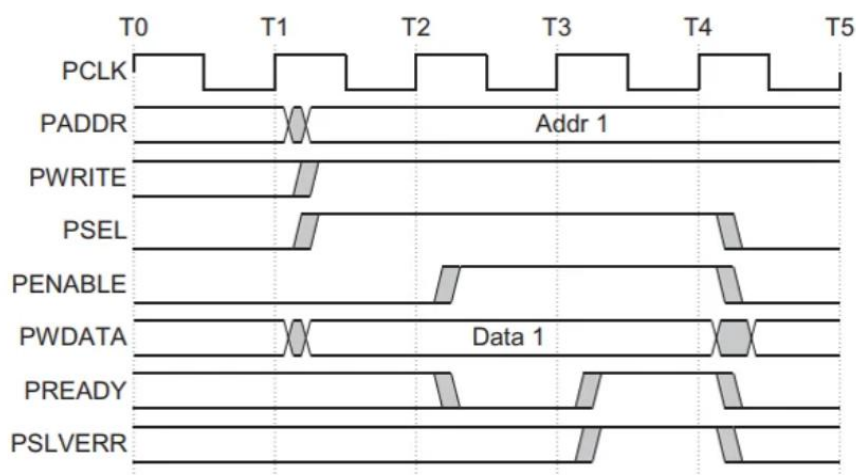
Hình 2.5. Quá trình đọc có trạng thái chờ

Giống như quá trình ghi có trạng thái chờ, sau khi kết thúc giai đoạn thiết lập ở T2, Master sẽ đợi Slave báo trạng thái sẵn sàng qua chân PREADY ở T4 để bắt đầu giai đoạn truy cập và đọc dữ liệu từ thanh ghi của Slave như đọc không có trạng thái chờ.

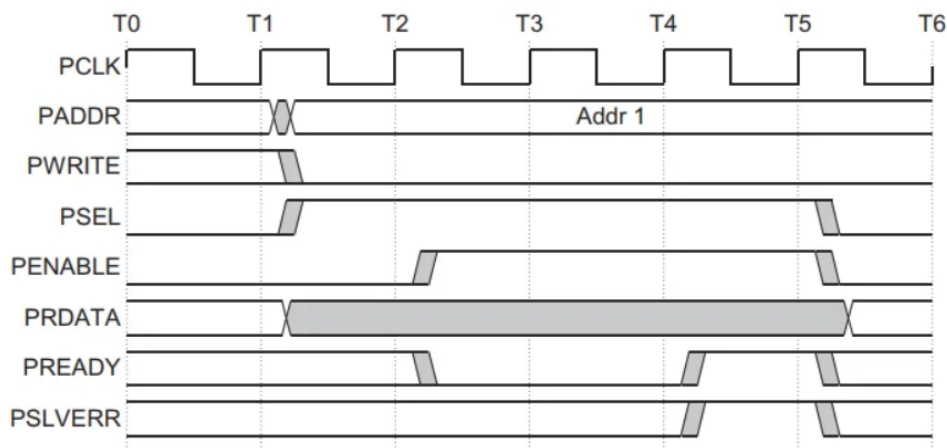
2.1.6 Phản hồi lỗi của giao thức APB3

Ở thế hệ thứ 3, APB được bổ sung cơ chế phản hồi lỗi qua chân PSLVERR. Tín hiệu PSLVERR được dùng để báo một giao dịch bị lỗi và không hoàn thành. Việc phản hồi lỗi có thể xảy ra ở cả quá trình đọc và ghi. Các lỗi thông thường liên quan đến việc đọc hoặc ghi vào vùng địa chỉ bị cấm hoặc cố tình ghi đè lên các thanh ghi quan trọng khi ngoại vi đang hoạt động.

Hoạt động của PSLVERR được mô tả theo hai hình 2.6 và 2.7:



Hình 2.6. Phản hồi lỗi trong quá trình ghi



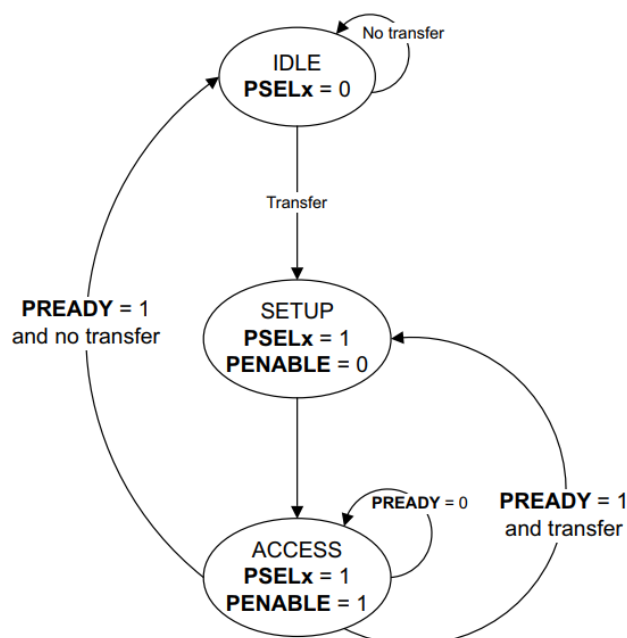
Hình 2.7. Phản hồi lỗi trong quá trình đọc

Khi phát sinh có lỗi xuất hiện trong quá trình giao dịch, tín hiệu PSLVERR được Slave kéo lên mức cao ở trong giai đoạn truy cập, khi mà cả PENABLE và PREADY đều ở mức cao. Quá trình phản hồi lỗi được thể hiện rõ qua 2 hình

trên. Với quá trình ghi ở hình 2.6, PSLVERR được bật lên ở T3 và ở T4 với quá trình đọc ở hình 2.7.

2.1.7 Trạng thái hoạt động của giao thức APB3

Giao thức APB3 hoạt động theo các trạng thái được mô tả bằng sơ đồ sau:



Hình 2.8. Sơ đồ trạng thái hoạt động của APB3

Từ sơ đồ trên giao thức APB3 hoạt động qua các trạng thái sau:

IDLE: Đây là trạng thái mặc định của APB. Trong trạng thái này, không có truyền dữ liệu diễn ra, các tín hiệu PSELx và PENABLE đều ở mức 0.

SETUP: Khi cần bắt đầu một giao dịch, Master sẽ đặt PSELx của Slave cần chọn lên mức cao. Trạng thái SETUP sẽ được giữ trong 1 chu kỳ để chuyển sang trạng thái tiếp theo.

ACCESS: Ở trạng thái này, PENABLE sẽ được Master đặt ở mức cao. Sau đó, Master sẽ theo dõi trạng thái Slave qua PREADY. Nếu PREADY ở mức thấp, Master sẽ chờ đến khi nó được bật lên mức cao sẽ tiến hành quá trình đọc hoặc ghi dữ liệu với thanh ghi tùy vào loại giao dịch và phản hồi lỗi nếu có. Cuối cùng, Master sẽ giữ PSEL và hạ PENABLE xuống mức thấp để quay lại SETUP cho giao dịch tiếp theo hoặc hạ cả hai xuống mức thấp để về trạng thái IDLE.

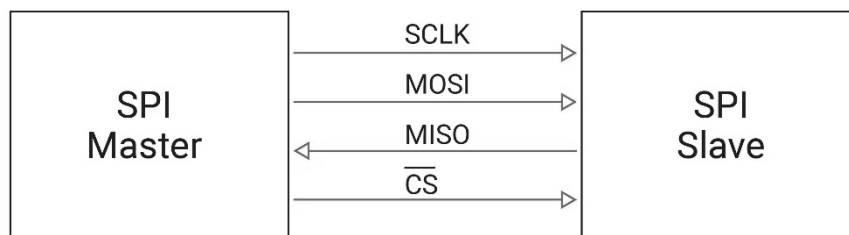
2.2 TỔNG QUAN VỀ CHUẨN GIAO TIẾP SPI

2.2.1 Giới thiệu về chuẩn giao tiếp SPI

SPI (Serial Peripheral Interface) là một chuẩn giao tiếp nối tiếp đồng bộ, hoạt động theo cơ chế song công toàn phần (full-duplex), cho phép truyền và nhận dữ liệu đồng thời với tốc độ cao. Chuẩn giao tiếp này sử dụng mô hình Master–Slave, trong đó thiết bị Master đóng vai trò điều khiển, tạo xung clock và quản lý quá trình truyền thông, còn các thiết bị Slave thực hiện trao đổi dữ liệu với Master thông qua tín hiệu chọn chip (CS/SS). Nhờ cơ chế truyền dữ liệu liên tục theo bit và không cần điều kiện bắt đầu hay kết thúc khung truyền, SPI đạt hiệu suất cao và độ trễ thấp, thường được sử dụng để kết nối vi điều khiển với các thiết bị ngoại vi như cảm biến, bộ nhớ Flash và các mạch tích hợp khác.

2.2.2 Các tín hiệu của SPI

Giao diện SPI phổ biến thường được cấu hình 4 tín hiệu cơ bản sau:



Hình 2.9. Giao diện SPI với Master và Slave

Serial Clock (SCLK): Tín hiệu xung clock do Master tạo ra nhằm đồng bộ cho quá trình truyền và lấy mẫu của Master lẫn Slave.

Chip Select (CS) / Slave Select (SS): Tín hiệu chọn Slave của Master nhằm xác định Slave cần thiết trong quá trình giao tiếp. Tín hiệu thường tích cực mức thấp.

Master Output Slave Input (MOSI): Đường truyền dữ liệu từ Master đến Slave.

Master Input Slave Output (MISO): Đường truyền dữ liệu từ Slave về Master.

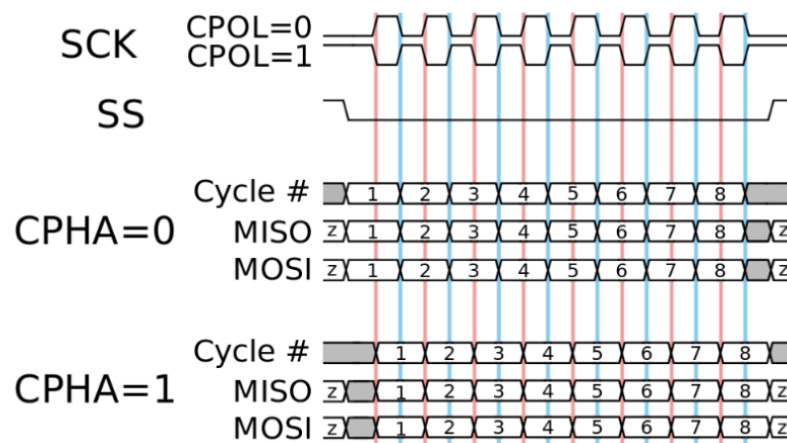
2.2.3 Nguyên lý hoạt động của SPI

Khi bắt đầu quá trình giao tiếp, Master kéo chân SS/CS xuống mức thấp để chọn Slave giao tiếp và bắt đầu tạo xung qua chân SCLK.

Sau đó, SPI master và SPI slave truyền và lấy mẫu dữ liệu nối tiếp thông qua MOSI và MISO, đồng bộ với SCLK theo các chế độ cấu hình đường truyền của SPI.

Khi một khung dữ liệu hoàn tất, Master dừng việc tạo SCLK và ngắt kết nối với Slave bằng cách đưa chân SS/CS lên mức cao. Ngoài ra, với chế độ truyền liên tục (continuous mode) của SPI, SS sẽ được Master giữ ở mức thấp trong suốt quá trình truyền nhiều khung liên tục cho đến khi kết thúc bằng cách đưa SS về mức cao.

SPI có 4 chế độ hoạt động bằng cách cấu hình hai tín hiệu CPHA và CPOL được thể hiện qua hình sau:



Hình 2.10. Dạng sóng truyền dữ liệu và lấy mẫu chế độ khác nhau của SPI

Trong quá trình hoạt động, việc truyền dữ liệu và lấy mẫu của cả Master lẫn Slave đều phải hoạt động trên các cạnh của SCLK như sau:

Chế độ 0 (CPOL=0, CPHA=0): SCLK nhận rồi ở mức thấp, dữ liệu thay đổi trong quá trình truyền xảy ra ở cạnh xuống SCLK và lấy mẫu dữ liệu ở cạnh lên.

Chế độ 1 (CPOL=0, CPHA=1): SCLK nhận rồi ở mức thấp, dữ liệu thay đổi trong quá trình truyền xảy ra ở cạnh lên SCLK và lấy mẫu dữ liệu ở cạnh xuống.

Chế độ 2 (CPOL=1, CPHA=0): SCLK nhận rồi ở mức cao, dữ liệu thay đổi trong quá trình truyền xảy ra ở cạnh xuống SCLK và lấy mẫu dữ liệu ở cạnh lên.

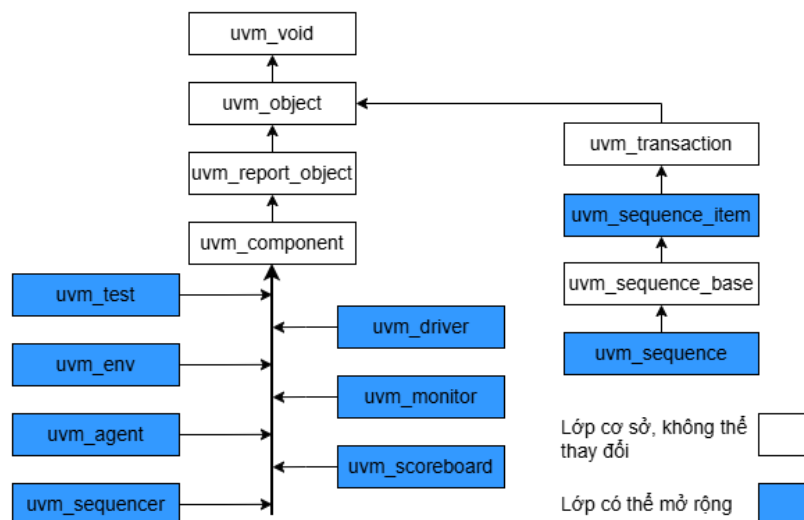
Chế độ 3 (CPOL=1, CPHA=1): SCLK nhận rồi ở mức thấp, dữ liệu thay đổi trong quá trình truyền xảy ra ở cạnh lên SCLK và lấy mẫu dữ liệu ở cạnh xuống.

2.3 PHƯƠNG PHÁP XÁC MINH UVM

2.3.1 Tổng quan về UVM

UVM (Universal Verification Methodology) là một phương pháp luận xác minh chuẩn được sử dụng trong công nghiệp bán dẫn nhằm xác minh các thiết kế số và hệ thống trên chip (SoC). UVM được xây dựng trên ngôn ngữ SystemVerilog và khai thác các đặc tính của lập trình hướng đối tượng (OOP) nhằm tạo ra các thành phần kiểm thử có tính mô-đun, dễ tái sử dụng và mở rộng trong các dự án tương lai. Bên cạnh đó, UVM sở hữu tính tiêu chuẩn hóa giúp người sử dụng nhanh chóng tiếp cận nguồn dữ liệu lớn mà không cần mất quá nhiều thời gian để bắt kịp công việc trong công ty. Ngoài ra, UVM hỗ trợ mô hình transaction giúp dễ dàng quản lý các nguồn dữ liệu vào và ra DUT giúp quá trình sửa lỗi diễn ra dễ dàng hơn.

UVM cung cấp một hệ thống các lớp cơ sở, từ đó người dùng có thể phát triển thông qua cơ chế kế thừa các lớp trước của lập trình hướng đối tượng nhằm đáp ứng yêu cầu của môi trường xác minh. Cấu trúc phân lớp của UVM được thể hiện qua sơ đồ sau:



Hình 2.11. Hệ thống phân cấp trong môi trường UVM

Các lớp UVM được tổ chức theo cấu trúc phân cấp. Khởi đầu với *uvm_void*, đây là lớp nền mang tính trừu tượng, không mang giá trị hay bất cứ phương thức cụ thể nào. Lớp *uvm_object* cung cấp các chức năng cơ bản như khởi tạo và sao chép đối tượng đồng thời hỗ trợ cấu hình cho môi trường xác minh. Trên cơ sở đó, cấu trúc UVM chia làm 2 nhánh là nhánh thành phần (component) và nhánh giao dịch (transaction).

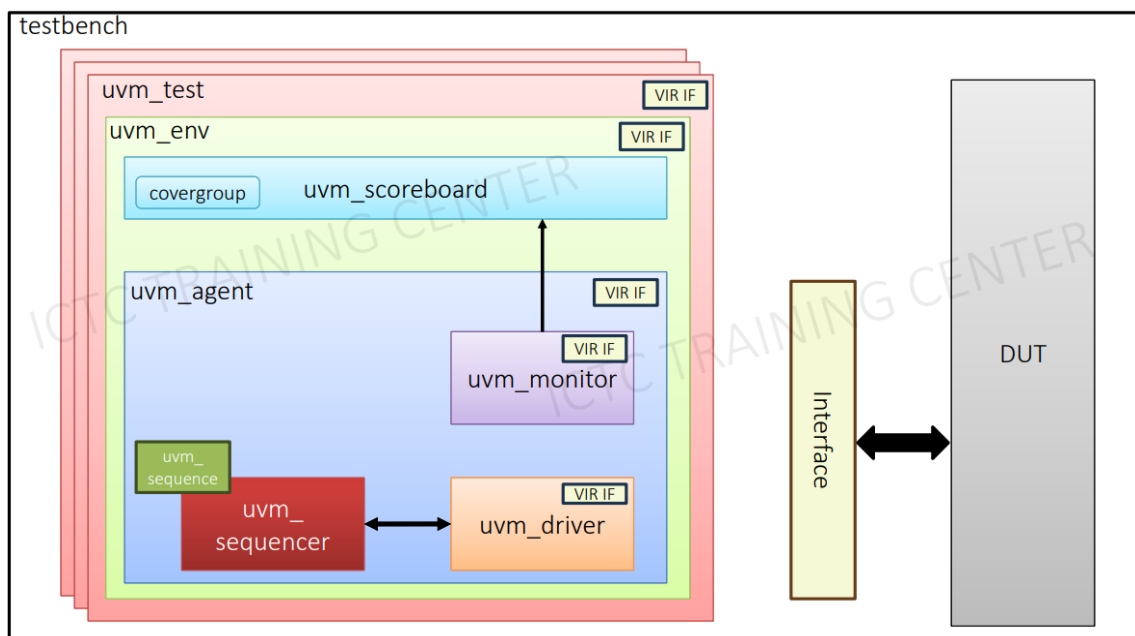
Ở nhánh thành phần, lớp *uvm_object_report* cung cấp cơ chế báo cáo và kiểm soát thông tin mô phỏng trong quá trình xác minh. Lớp này định nghĩa 4 mức báo cáo bao gồm: *info*, *warning*, *error* và *fatal* ứng với từng mức độ nghiêm trọng khác nhau xảy ra trong quá trình mô phỏng. Các thông báo này ngoài việc thông báo những lỗi phát sinh hay thông tin những dữ liệu ra vào DUT cho quá trình sửa lỗi mà còn có thể dừng mô phỏng khi xảy ra những lỗi nghiêm trọng khiến môi trường bị treo. Kế thừa *uvm_object* và *uvm_report_object*, lớp *uvm_component* đóng vai trò là khung kiến trúc chính cho các lớp thành phần của testbench như sequencer, driver, monitor, agent, environment, scoreboard... trong môi trường kiểm thử. Đồng thời, lớp này cung cấp các cơ chế thuận tiện trong quá trình xây dựng môi trường như:

- **Phân cấp (Hierarchy):** Hỗ trợ cấu trúc phân cấp, trong đó mỗi thành phần có thể có các thành phần con, tạo thành cây phân cấp của testbench và giúp quản lý phạm vi hoạt động rõ ràng.
- **Phân pha (Phasing):** Cho phép các thành phần tham gia vào các pha chuẩn của UVM như build, connect và run để thực hiện các chức năng tương ứng theo vòng đời mô phỏng.
- **Báo cáo (Reporting):** Cung cấp cơ chế báo cáo thông qua hệ thống thông báo của *uvm_report* nhằm ghi thông tin, cảnh báo và lỗi trong quá trình mô phỏng.
- **Tạo đối tượng (Factory):** Hỗ trợ tạo các thành phần một cách linh hoạt và cho phép thay thế cách cài đặt khi cần thiết mà không phải chỉnh sửa mã nguồn ban đầu.

Ở nhánh giao dịch, *uvm_transaction* kế thừa lớp *uvm_object* giúp mô tả một giao dịch dữ liệu được trao đổi giữa các thành phần của testbench như sequencer, driver và monitor. Lớp này đóng vai trò mang thông tin trong quá trình xác minh, giúp chuẩn hóa cách biểu diễn và xử lý dữ liệu giao dịch. Nó không tham gia vào cấu trúc phân cấp hay vòng đời mô phỏng như *uvm_component*.

2.3.2 Cấu trúc môi trường UVM

Áp dụng cơ chế kế thừa và hướng đối tượng, môi trường kiểm thử UVM được xây dựng theo kiến trúc phân cấp (hierarchy) nhằm đảm bảo tính mô-đun hóa và khả năng tái sử dụng cao. Sơ đồ tổ chức các thành phần trong một testbench tiêu chuẩn được thể hiện như sau:



Hình 2.12. Sơ đồ tổ chức UVM testbench tiêu chuẩn

Chức năng và nhiệm vụ cụ thể của từng thành phần trong sơ đồ trên được mô tả chi tiết theo từng phần như sau:

2.3.2.1 Lớp testbench

Testbench còn được gọi là UVM Testbench Top, được tổ chức là một mô-đun tĩnh nằm ở lớp cao nhất. Đây được xem là vùng chứa để lưu trữ các thành phần cần thiết của quá trình mô phỏng và trở thành gốc cho hệ thống phân cấp

trong môi trường. Ngoài ra, mô-đun đóng vai trò kết nối các nhóm tín hiệu vật lý từ DUT và thiết lập các giao diện ảo từ đó và phân phối nó đến các lớp bên dưới thông qua cơ chế *uvm_config_db*. Điều này giúp cho các thành phần bên dưới có quyền truy cập và điều khiển tín hiệu phần cứng. Cuối cùng, testbench kích hoạt cơ chế phân pha nhằm xây dựng cấu trúc môi trường và thực thi các kịch bản kiểm thử.

2.3.2.2 Lớp *uvm_test*

Đây là lớp đứng đầu trong cấu trúc phân cấp của nhánh thành phần, chịu trách nhiệm xây dựng khung sườn cho môi trường xác thực (*uvm_env*) và thiết lập các cấu hình ban đầu. Lớp này tiếp nhận các giao diện ảo từ testbench và phân phối chúng đến các thành phần bên dưới. Tuy nhiên, chức năng cốt lõi của lớp là định nghĩa, khởi tạo, kích hoạt các tham số cấu hình môi trường cùng kịch bản kiểm thử và điều phối chúng đến các thành phần bên dưới để đảm bảo tín hiệu được gửi đến DUT một cách chính xác qua các giao diện ảo đã cung cấp.

2.3.2.3 Lớp *uvm_env*

Lớp môi trường (*uvm_test*) là vùng chứa trung tâm của kiến trúc kiểm thử. Lớp có nhiệm vụ xây dựng và kết nối các thành phần có tính tái sử dụng như một hoặc nhiều tác nhân (*uvm_agent*), bảng điểm (*uvm_scoreboard*), hoặc các thành phần nâng cao như bộ kiểm tra (*checker*) và bộ giám sát (*top_level monitor*). Từ đó, *uvm_env* điều phối các tham số cấu hình và giao diện ảo cho các thành phần bên trong nó. Điểm ưu việt của lớp này là khả năng phân cấp linh hoạt, cho phép lồng ghép nguyên vẹn các môi trường con từ mức khối (block level) vào hệ thống phức tạp (system level), qua đó tối ưu hóa việc tái sử dụng tài nguyên và kịch bản kiểm thử sẵn có.

2.3.2.4 Lớp *uvm_agent*

Lớp tác nhân hay *uvm_agent* đại diện tiêu chuẩn cho một giao thức cụ thể như APB, AHB, SPI, UART, I2C... Lớp này đóng gói các hoạt động điều khiển và giám sát các giao thức trên trong một thực thể duy nhất. Trong một môi

trường *uvm_env* có thể tồn tại nhiều *uvm_agent* đại diện cho các thực thể khác nhau. Bên trong nó, các thành phần chức năng như *uvm_sequencer*, *uvm_driver*, *uvm_monitor* được kết nối chặt chẽ qua giao diện mức giao dịch (Transaction Level Modeling - TLM) cho phép chúng trao đổi dữ liệu dưới dạng các gói tin trừu tượng thay vì xử lý từng dây tín hiệu vật lý phức tạp. Người dùng có thể cấu hình *uvm_agent* ở mức chủ động (ACTIVE) để trực tiếp điều khiển tín hiệu vào thiết kế qua giao thức ảo mà các lớp trên cung cấp hoặc ở mức thụ động (PASSIVE) chỉ quan sát và thu thập dữ liệu xảy ra trên đường truyền.

2.3.2.5 Lớp *uvm_sequencer*

Đây là nơi các tham số, các thuộc tính cho gói giao dịch (*uvm_transaction*) và kịch bản kiểm thử ở lớp *uvm_test* được tổng hợp và tạo ra các gói dữ liệu giao dịch dưới dạng *uvm_sequence* kế thừa các thuộc tính của *uvm_transaction*. Từ đó, lớp này sẽ điều tiết tuần tự những gói dữ liệu đó một cách tuần tự theo giao thức bắt tay với *uvm_driver*.

2.3.2.6 Lớp *uvm_driver*

Đây là thành phần đóng vai trò trung gian giữa *uvm_sequencer* và DUT, chịu trách nhiệm chuyển đổi các *uvm_sequence* trừu tượng thành các tín hiệu mức chân cụ thể. Từ đó, *uvm_driver* sử dụng giao diện ảo (virtual interface) nhận được truyền từ *uvm_agent* để điều khiển các tín hiệu của DUT, đồng thời đảm bảo việc truyền dữ liệu tuân thủ đúng giao thức mà agent chứa nó thể hiện.

2.3.2.7 Lớp *uvm_monitor*

Là một thành phần luôn hoạt động bất chấp trạng thái của agent chứa nó, *uvm_monitor* quan sát hoạt động của DUT thông qua giao diện ảo (virtual interface) được nhận từ agent chứa nó. Nó thu thập các tín hiệu mức chân, chuyển đổi chúng thành các *uvm_transaction* ở mức trừu tượng và gửi đến các thành phần phân tích như *uvm_scoreboard* để phân tích và so sánh.

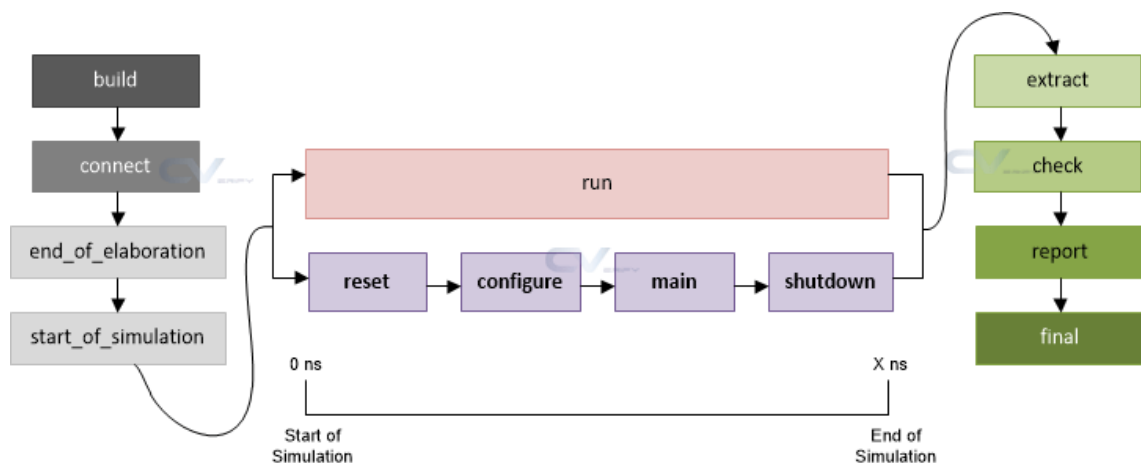
2.3.2.8 Lớp `uvm_scoreboard`

Đây là nơi xử lý các dữ liệu trừu tượng dưới dạng `uvm_transaction` được gửi về từ các agent khác nhau trong môi trường `uvm_env`. Các dữ liệu đó sẽ được so sánh với các kết quả mong đợi nhằm kiểm tra và đánh giá tính đúng đắn trong hoạt động của DUT. Bên cạnh đó, `uvm_scoreboard` có thể tích hợp covergroup để thu thập độ bao phủ chức năng (functional coverage), giúp xác định mức độ các trạng thái, giá trị và kịch bản đã được kiểm thử, từ đó đánh giá chất lượng tổng thể của môi trường xác minh.

2.3.3 Các cơ chế trong môi trường UVM

2.3.3.1 Cơ chế phân pha (UVM Phasing)

Cơ chế phân pha được xem là xương sống trong kiến trúc UVM. Nó giúp kiểm soát trình tự và nhịp hoạt động của toàn bộ môi trường xác minh. Nhờ cơ chế này, các thành phần trong testbench được phối hợp một cách nhất quán xuyên suốt quá trình mô phỏng. Cơ chế pha của UVM được phân cấp như sau:



Hình 2.13. Cấu trúc phân cấp cơ chế pha của UVM

Dựa trên cấu trúc trên, cơ chế phân pha được chia thành ba giai đoạn chính:

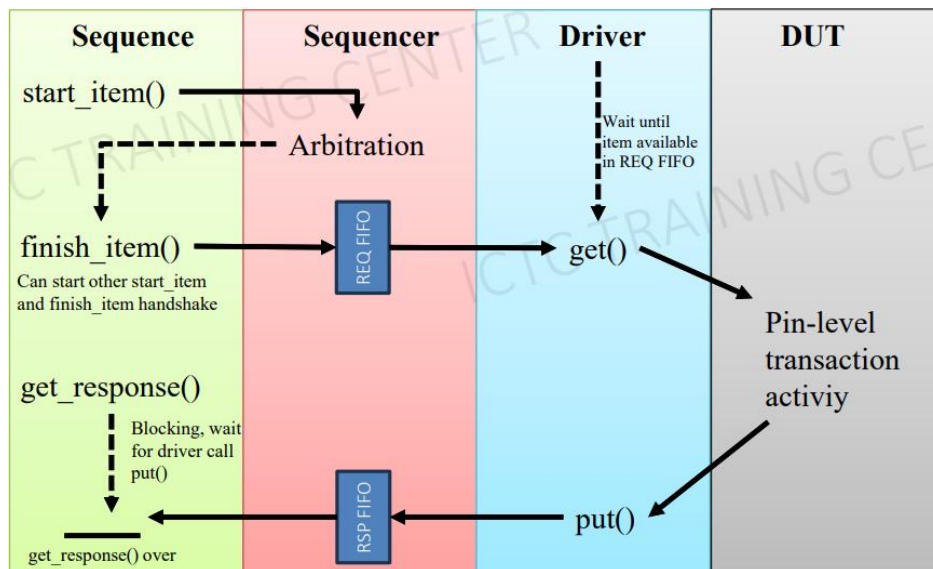
- Giai đoạn khởi tạo: là nhóm pha từ *build* đến *start_of_simulation*. Giai đoạn này tập trung vào việc xây dựng các thành phần, thiết lập cấu hình ban đầu và kết nối chúng để hình thành cấu trúc testbench của UVM.
- Giai đoạn mô phỏng: diễn ra trong *run_phase* đồng thời chạy song song với các pha từ *reset* đến *shutdown*. Đây là pha hoạt động chính được bắt

đầu ở lớp testbench. Tại đây, môi trường thực hiện điều khiển và quan sát phản hồi của DUT theo các kịch bản kiểm thử trong thời gian mô phỏng.

- Giai đoạn kết thúc: bao gồm các *extract*, *check*, *report*, *final* đảm nhận nhiệm vụ thu thập dữ liệu, đánh giá kết quả và phản hồi báo cáo sau khi hoàn tất mô phỏng.

2.3.3.2 Cơ chế bắt tay giữa sequencer và driver

Trong cơ chế này, UVM cung cấp các phương thức tích hợp để điều phối quá trình trao đổi dữ liệu kiểm thử giữa sequencer và driver một cách trình tự. Cơ chế này đảm bảo quá trình điều tiết các gói dữ liệu một cách tuần tự giữa hai thành phần này, đặc biệt trong các giao thức tuần tự như UART, SPI, I2C... Quy trình bắt tay được thể hiện trong sơ đồ sau:



Hình 2.14. Cơ chế bắt tay giữa sequencer và driver

Quá trình bắt đầu khi *uvm_test* bắt đầu khởi tạo sequence. Sequence sẽ yêu cầu tạo một gói dữ liệu mới với *start_item()*. Lúc này, sequencer sẽ xem xét tất cả các yêu cầu mà đưa ra quyết định chọn theo các chính sách được cấu hình như độ ưu tiên (priority), lập lịch vòng (round robin) hoặc ngẫu nhiên (random). Khi yêu cầu không được chấp nhận, nó sẽ bị chặn ở *start_item()* và sẽ chờ cho đến khi được chấp nhận. Khi đó, sequence sẽ thông báo *finish_item()* đồng thời đặt dữ liệu đó vào hàng đợi yêu cầu (REQ FIFO).

Trong lúc đó, driver sẽ chờ cho đến khi có dữ liệu trong hàng chờ và lấy nó với *get()*. Khi đã lấy thành công gói dữ liệu từ sequencer, driver tiến hành chuyển các dữ liệu trừu tượng đó sang tín hiệu điều khiển DUT. Khi đã hoàn thành, driver tạo một gói dữ liệu phản hồi và đặt vào hàng chờ phản hồi (RSP FIFO) nhằm báo cáo đã hoàn thành.

Sequence sẽ liên tục gọi *get_response()* cho đến khi có gói phản hồi trong RSP FIFO để hoàn thành chu kỳ bắt tay giữa sequencer và driver.

2.3.3.3 Cơ chế mô hình hoá giao dịch (Transaction-Level Modeling - TLM)

Transaction-Level Modeling (TLM) trong UVM là cơ chế truyền thông ở mức trừu tượng cao, cho phép các thành phần trong môi trường xác minh trao đổi dữ liệu dưới dạng các giao dịch thay vì thao tác trực tiếp trên từng tín hiệu mức trên các chân. Cách tiếp cận này làm cho quá trình xây dựng và phân tích testbench trở nên trực quan hơn, đồng thời tạo nền tảng cho việc kết nối linh hoạt giữa các thành phần UVM.



Hình 2.15. Cơ chế mô hình hoá giao dịch (TLM)

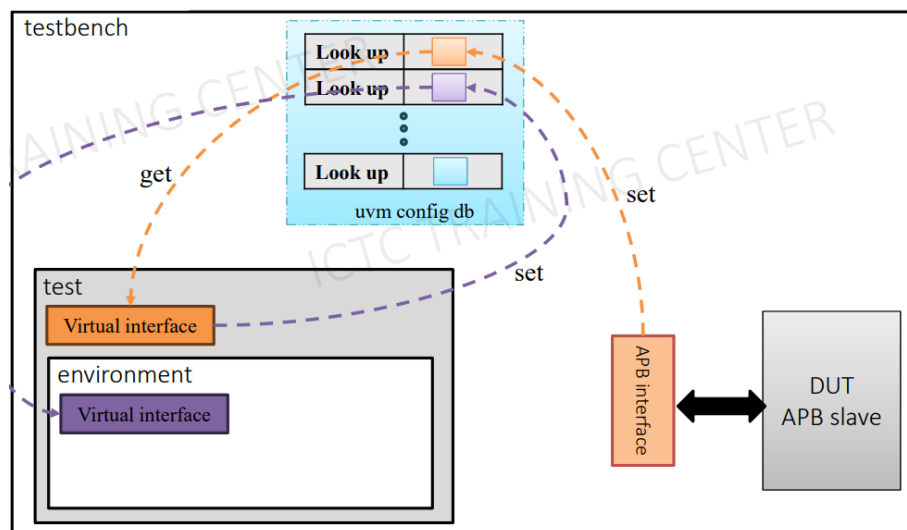
Như đã thể hiện trong các sơ đồ, UVM không giao tiếp trực tiếp với nhau mà thông qua cơ chế TLM với các phần tử kết nối như *port*, *export* và *import*. Thành phần phát dữ liệu sử dụng *port* để gửi giao dịch, trong khi thành phần nhận cung cấp *export/import* để tiếp nhận và xử lý dữ liệu tương ứng. Nhờ cách kết nối này, mỗi component chỉ cần tuân theo giao diện TLM đã định mà không phụ thuộc vào chi tiết cài đặt của component còn lại, từ đó tăng tính độc lập, dễ thay thế và nâng cao khả năng tái sử dụng của toàn bộ môi trường xác minh UVM.

2.3.3.4 Cơ chế đăng kí Factory

Trong kiến trúc UVM, cơ chế Factory đóng vai trò là cơ chế trung tâm điều phối việc khởi tạo các thành phần (components) và đối tượng (objects). Factory cung cấp tính năng ghi đè (override), cho phép người dùng linh hoạt thay thế một lớp đối tượng mặc định bằng một lớp dẫn xuất khác ngay trong thời gian chạy (run-time). Nhờ đó, cấu trúc testbench có thể thay đổi hành vi động mà không cần can thiệp vào mã nguồn gốc, tối ưu hóa tính tái sử dụng của môi trường kiểm tra.

2.3.3.5 Cơ chế *uvm_config_db*

Cơ chế *uvm_config_db* được sử dụng trong việc lưu trữ và truy xuất những thông tin dùng chung giữa các thành phần với nhau. Thông qua cơ chế này, việc chia sẻ những cấu hình được thực hiện một cách thống nhất, có kiểm soát thay vì truyền trực tiếp qua các tham số khởi tạo.

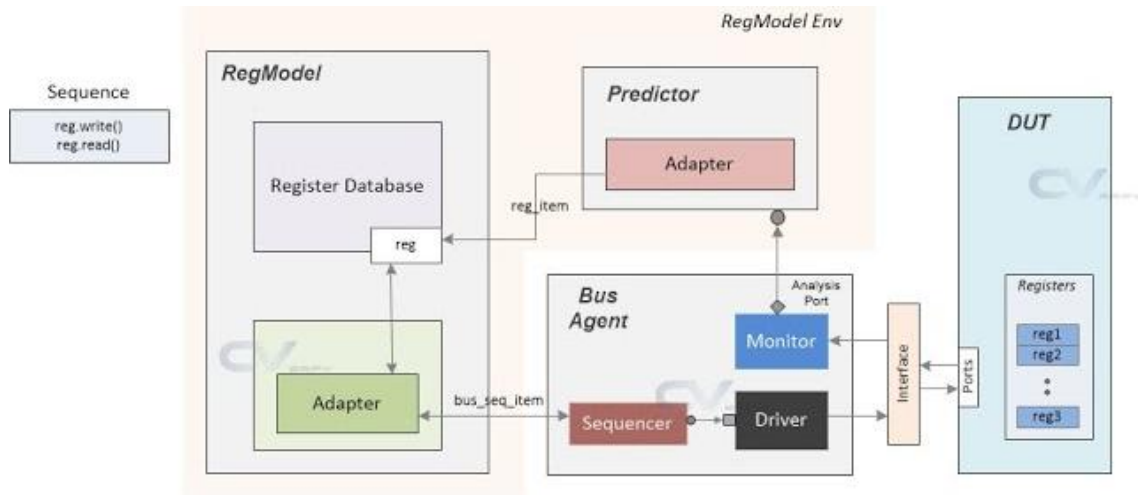


Hình 2.16. Cơ chế *uvm_config_db*

Cơ chế *uvm_config_db* cũng có thể xem là một kho cấu hình trung tâm, nơi mà các thông tin cấu hình được các lớp nằm cao trong cấu trúc phân tầng đặt vào qua cơ chế *set()*. Từ đó, các thành phần mục tiêu ở lớp dưới sẽ truy xuất các cấu hình tương ứng bằng cơ chế *get()* thông qua cơ chế tra cứu theo tên và mục tiêu. Từ đó, *uvm_config_db* giúp việc chia sẻ dữ liệu giữa các thành phần trở nên rõ ràng, linh hoạt và dễ mở rộng trong môi trường xác minh UVM.

2.3.3.6 Cơ chế Register Abstraction Layer (RAL)

Register Abstraction Layer (RAL) là cơ chế trong UVM cho phép mô hình hóa và truy cập các thanh ghi của DUT ở mức trừu tượng, tách biệt việc kiểm chứng chức năng thanh ghi khỏi chi tiết của giao thức bus. Thay vì thao tác trực tiếp trên tín hiệu mức chân, testbench làm việc với các thanh ghi thông qua một register model thống nhất, giúp việc kiểm tra trở nên có hệ thống và dễ mở rộng.



Hình 2.17. Cơ chế Register Abstraction Layer (RAL)

Trung tâm của cơ chế RAL là mô hình thanh ghi (register model hay regmodel). Đây là nơi đại diện cho toàn bộ hệ thống thanh ghi của DUT dưới dạng cấu trúc trừu tượng. Mô hình này mô tả cách các thanh ghi được tổ chức, phân chia và ánh xạ địa chỉ, đóng vai trò là điểm tham chiếu thống nhất để môi trường xác minh thực hiện các thao tác kiểm tra liên quan đến thanh ghi.

Với mỗi thanh ghi trong mô hình thanh ghi được quản lý thông qua hai giá trị là giá trị mong muốn (desired value) và giá trị ánh xạ (mirror value). Giá trị mong muốn thể hiện trạng thái kỳ vọng của thanh ghi trong kịch bản kiểm tra hiện tại trong khi giá trị ánh xạ phản ánh giá trị thực tế của thanh ghi trong DUT. Điều này cho phép môi trường có thể đánh giá trạng thái thanh ghi một cách nhất quán trong cả quá trình mô phỏng.

Các thao tác đọc và ghi thanh ghi chủ yếu được thực hiện thông qua cơ chế truy cập gián tiếp, trong đó các yêu cầu từ mô hình thanh ghi được chuyển đổi bởi *adapter* thành các giao dịch bus phù hợp và được thực thi thông qua *bus*

agent để tác động lên DUT. Ở chiều ngược lại, *monitor* quan sát các giao dịch thực tế trên bus và chuyển thông tin này đến *predictor*, từ đó cập nhật giá trị *mirror* trong mô hình thanh ghi, đảm bảo trạng thái logic luôn phản ánh đúng hành vi của DUT. Ngoài ra, RAL cũng hỗ trợ truy cập trực tiếp thông qua cấu trúc phân cấp của thiết kế, chủ yếu phục vụ cho mục đích khởi tạo nhanh hoặc gỡ lỗi.

Từ đó, RAL góp phần chuẩn hóa quy trình xác minh thanh ghi, giảm độ phức tạp khi mở rộng testbench và tạo điều kiện thuận lợi cho việc tái sử dụng môi trường xác minh trong các hệ thống UVM quy mô lớn.

CHƯƠNG 3

THIẾT KẾ HỆ THỐNG

3.1 YÊU CẦU HỆ THỐNG

3.1.1 Yêu cầu thiết kế IP SPI master giao tiếp qua APB3

Trước khi đi vào thiết kế chi tiết, cần xác định rõ các yêu cầu hệ thống nhằm đảm bảo IP SPI giao tiếp APB hoạt động đúng chức năng. Trong quá trình thiết kế, hệ thống cần đáp ứng các yêu cầu:

- IP phải có khả năng giao tiếp qua giao thức APB3, và tương thích với hầu hết các tần số hoạt động thường dùng của APB, hỗ trợ phản hồi lỗi và hoạt động với không trạng thái chờ (no wait state).
- Hệ thống cần đảm bảo đầy đủ các tín hiệu cơ bản của SPI và hoạt động đúng với chức năng của một SPI master.
- Hỗ trợ chế độ truyền liên tục (continuous mode) và giao tiếp với 4 Slave.
- Hỗ trợ cấu hình hệ số chia SCLK, cấu hình chế độ CPOL/CPHA, cấu hình độ rộng khung dữ liệu 8/16 bit và điều khiển hệ thống ngắt.
- Hỗ trợ tín hiệu ngắt với các trạng thái FIFO.

3.1.2 Yêu cầu thiết kế môi trường kiểm thử UVM

Môi trường kiểm thử UVM được xây dựng nhằm kiểm tra IP hoạt động đúng với tất cả các cấu hình theo đặc tả. Các yêu cầu chính bao gồm:

- **Kiểm thử cấu hình qua APB:** Xây dựng APB Verification IP (APB VIP) đóng vai trò là APB master truy cập DUT thông qua bus APB kết hợp với cơ chế UVM RAL (Register Abstraction Layer) cho việc cấu hình và kiểm tra các thanh ghi và hành vi của tín hiệu ngắt.

- **Kiểm thử giao tiếp SPI:** Xây dựng SPI Verification IP (SPI VIP) đóng vai trò là SPI slave nhằm điều phối dữ liệu gửi qua MISO và nhận tín hiệu từ SPI, kiểm tra tần số SCLK và quá trình chọn Slave qua 4 chân SS.
- **Kiểm thử tích hợp APB–SPI:** Kết hợp APB VIP và SPI VIP để xác minh tính đúng đắn của dữ liệu trong quá trình hoạt động của IP với từng chế độ cấu hình và tần số tạo ra.

3.2 THIẾT KẾ HỆ THỐNG PHẦN CỨNG

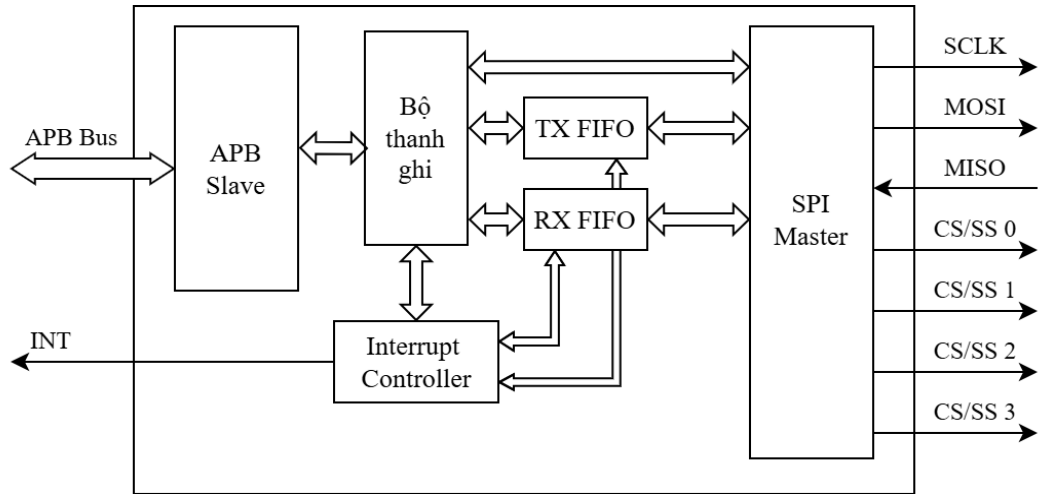
3.2.1 Chức năng hệ thống

IP SPI là bộ điều khiển giao tiếp đồng bộ song công hoạt động ở chế độ Master, được thiết kế để tích hợp vào các hệ thống dựa trên chuẩn AMBA với giao diện APB. IP cung cấp khả năng truyền nhận dữ liệu theo chuẩn SPI với khả năng cấu hình linh hoạt. Các chức năng chính bao gồm:

- **Bộ điều khiển SPI Master:** Cho phép truyền nhận dữ liệu nối tiếp đồng bộ theo chuẩn SPI với tính năng như:
 - Hỗ trợ cấu hình khung dữ liệu 8 bit hoặc 16 bit.
 - Truyền, nhận dữ liệu MSB trước.
 - Hỗ trợ 4 mode SPI thông qua cấu hình PHASE và POLARITY.
 - Hỗ trợ truyền dữ liệu liên tục hoặc không liên tục.
- Không gian thanh ghi được địa chỉ hóa với 12 bit.
- **Giao diện bus APB:** Cho phép cấu hình và điều khiển bộ điều khiển SPI thông qua giao diện AMBA APB3 với tính năng phát hiện và báo lỗi khi có truy cập bus không hợp lệ hoặc giá trị cấu hình sai.
- **Clock hệ thống và Reset:** Hoạt động hầu hết tần số thường gặp của APB và reset bất đồng bộ tích cực mức thấp.
- **Bộ chia xung clock SPI:** Tạo xung SCLK cho bộ điều khiển SPI từ clock hệ thống với hệ số chia có thể cấu hình được từ 0 - 255 với công thức (3.1).
- **Hỗ trợ ngắt (Interrupt):** Tạo ra tín hiệu ngắt dựa trên các điều kiện có thể cấu hình, có thể bật tắt thông qua các thanh ghi điều khiển.

3.2.2 Sơ đồ khối hệ thống

IP SPI giao tiếp APB sẽ bao gồm nhiều khối chính được kết nối với nhau. Sơ đồ khối tổng quát của hệ thống được thể hiện theo hình sau:



Hình 3.1. Sơ đồ khối tổng quát của IP SPI giao tiếp APB

Dựa vào chức năng đã được đề cập, các khối chính của IP SPI bao gồm:

Khối APB Slave: Đóng vai trò là cầu nối giữa bus APB và IP SPI. Chịu trách nhiệm tiếp nhận các tín hiệu điều khiển, địa chỉ và dữ liệu từ bus APB, sau đó truyền đến bộ thanh ghi bên trong IP. Đồng thời, APB slave cũng trả dữ liệu đọc từ các thanh ghi về cho APB master trên hệ thống. Nhờ đó, APB master có thể cấu hình hoạt động và giám sát trạng thái của IP SPI thông qua bus APB.

Bộ thanh ghi: Là nơi lưu trữ các thông tin cấu hình và trạng thái của IP. Thông qua bộ thanh ghi, người dùng có thể thiết lập mọi thao tác cấu hình như: chọn chế độ CPOL/CPHA, cấu hình tốc độ xung SCLK, kích hoạt hoặc vô hiệu hóa ngắt, điều khiển chọn SPI slave, đồng thời lưu giá trị cần gửi và đọc các dữ liệu đã nhận.

Khối TX FIFO (Transmit FIFO): Hoạt động như một bộ đệm hàng đợi cho dữ liệu truyền đi. Khi APB master ghi dữ liệu qua APB vào SPI, dữ liệu được lưu tạm trong TX FIFO trước khi SPI master sử dụng cho quá trình truyền, đồng thời thông báo trạng thái trống hoặc đầy.

Khối RX FIFO (Receive FIFO): Tương tự như TX FIFO, RX FIFO là bộ đệm dữ liệu lưu những giá trị đã nhận được qua MISO ở SPI master và chờ được

đọc ra bởi APB master, đồng thời cũng thông báo trạng thái trống và đầy cho các khối khác.

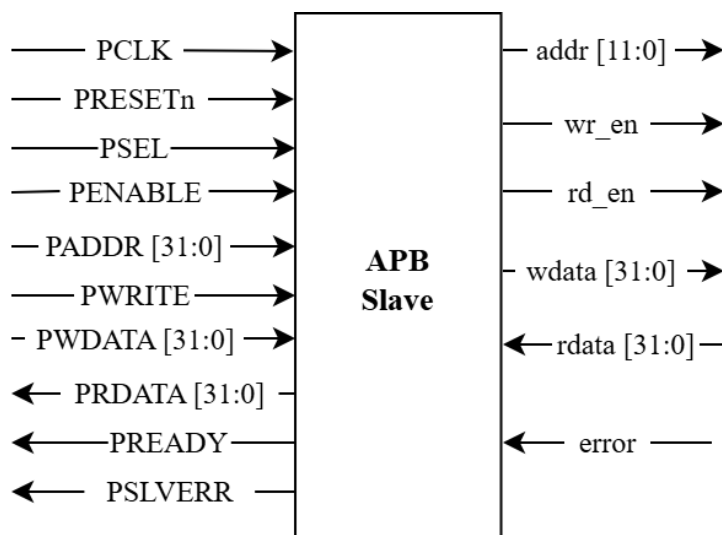
Khối Interrupt Controller: Chịu trách nhiệm quản lý và phát sinh ngắt cho hệ thống với các trạng thái của 2 FIFO, được bật tắt và loại ngắt qua cấu hình ban đầu.

Khối SPI Master: Là thành phần trung tâm thực hiện quá trình truyền và nhận dữ liệu theo chuẩn SPI. Khối này hoạt động theo cấu hình được thiết lập từ bộ thanh ghi, nó tạo ra xung clock SCLK, đồng bộ dữ liệu truyền trên chân MOSI và nhận dữ liệu trên chân MISO theo cấu hình CPOL/CPHA. Ngoài ra, khối này còn quản lý tín hiệu chọn Slave CS/SS để đảm bảo việc giao tiếp chính xác với các thiết bị đích.

3.2.3 Thiết kế khối APB Slave

3.2.3.1 Sơ đồ khối

Khối APB Slave là giao diện trung gian giữa bus APB và IP SPI, có nhiệm vụ truyền nhận dữ liệu, tín hiệu điều khiển, địa chỉ giữa bus APB và IP SPI. Sơ đồ khối tổng quát của khối APB Slave được thể hiện như hình 3.2:



Hình 3.2. Sơ đồ khối tổng quát khối APB Slave

Chi tiết các chân tín hiệu của khối APB Slave được mô tả theo bảng sau:

Bảng 3.1. Danh sách các chân tín hiệu của khối APB Slave

Tên tín hiệu	Độ rộng (bit)	Loại tín hiệu	Mô tả
PCLK	1	Tín hiệu vào	Tín hiệu xung clock của hệ thống
PRESETn	1	Tín hiệu vào	Tín hiệu reset hệ thống, tích cực mức thấp
PSEL	1	Tín hiệu vào	Tín hiệu lựa chọn
PENABLE	1	Tín hiệu vào	Tín hiệu cho biết các chu kỳ tiếp theo trong quá trình giao dịch
PADDR	32	Tín hiệu vào	Địa chỉ thanh ghi cần truy cập
PWRITE	1	Tín hiệu vào	Cho biết loại giao dịch: • 0: đọc • 1: ghi
PWDATA	32	Tín hiệu vào	Dữ liệu cho giao dịch ghi
PRDATA	32	Tín hiệu ra	Dữ liệu trả về cho giao dịch đọc
PREADY	1	Tín hiệu ra	Thông báo trạng thái sẵn sàng của IP
PSLVERR	1	Tín hiệu ra	Thông báo lỗi xảy ra trong quá trình giao dịch
addr	12	Tín hiệu ra	Thông báo địa chỉ thanh ghi cần truy cập
wr_en	1	Tín hiệu ra	Cho phép ghi dữ liệu vào thanh ghi
rd_en	1	Tín hiệu ra	Cho phép đọc thanh ghi
wdata	32	Tín hiệu ra	Dữ liệu cần ghi vào thanh ghi
rdata	32	Tín hiệu vào	Dữ liệu đọc từ thanh ghi
error	1	Tín hiệu vào	Thông báo lỗi từ thanh ghi

Bảng 3.1 mô tả các tín hiệu của khối APB Slave, bao gồm tín hiệu điều khiển, dữ liệu và trạng thái. Các tín hiệu đầu vào từ bus APB như clock, reset, địa chỉ và dữ liệu dùng để thiết lập và điều khiển giao tiếp; trong khi các tín hiệu đầu ra cung cấp dữ liệu phản hồi, báo trạng thái sẵn sàng hoặc lỗi, cũng như điều

khuyến truy cập bộ thanh ghi. Nhờ đó, khối APB Slave thực hiện trao đổi dữ liệu với bus APB một cách chính xác và tuân theo giao thức.

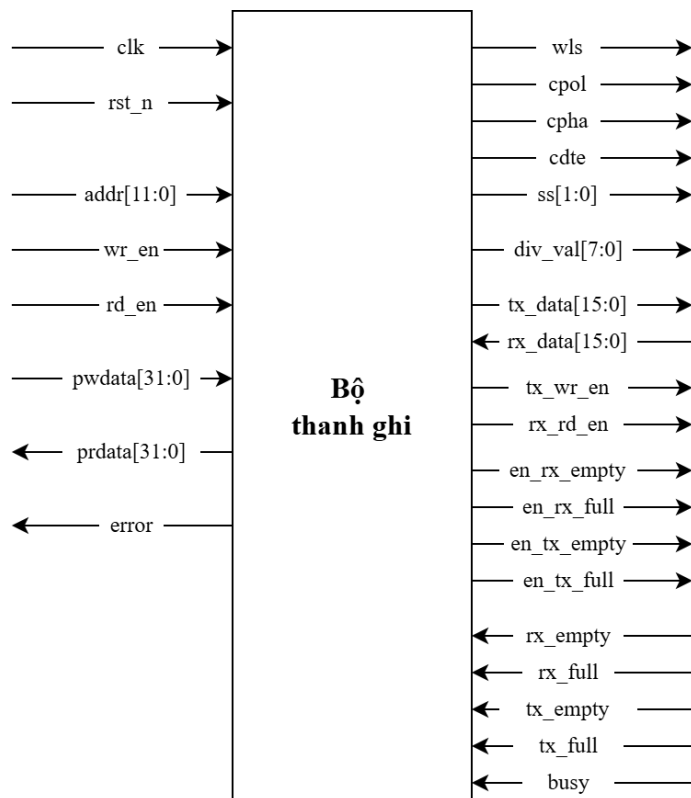
3.2.3.2 Cách hoạt động

APB slave đóng vai trò là trung gian giữa APB master và hệ thống thanh ghi điều khiển phía trong IP. APB slave tuân thủ quy trình giao dịch của APB. Khi trong giai đoạn thiết lập (Setup phase) với $PSEL = 1$, khối kiểm tra địa chỉ thanh ghi cần truy cập và loại giao dịch. Nếu mọi thứ đều hợp lệ, khối tiến hành giao tiếp với thanh ghi qua chân wr_en để yêu cầu ghi dữ liệu hoặc rd_en để yêu cầu đọc dữ liệu từ thanh ghi có địa chỉ được cung cấp từ đầu trong giai đoạn truy cập (Access phase). Cũng tại giai đoạn này, nếu phát hiện địa chỉ không hợp lệ khác vùng từ $0x4000_2000$ đến $0x4000_2014$ hoặc có tín hiệu lỗi từ khối thanh ghi qua chân *error*, APB slave sẽ bật PSLVERR và hủy giao dịch. Vì chỉ hỗ trợ chế độ không trạng thái chờ (no wait state) nên PREADY được giữ cố định ở mức cao.

3.2.4 Bộ thanh ghi

3.2.4.1 Sơ đồ khối

Để cấu hình và điều khiển hoạt động của IP SPI, các thanh ghi được ánh xạ vào không gian địa chỉ nhớ với các chức năng khác nhau. Từ đó, xây dựng thành một bộ thanh ghi có chức năng lưu trữ các thông tin cấu hình và trạng thái của IP. Sơ đồ khối tổng quát của bộ thanh ghi được thể hiện như hình 3.3:



Hình 3.3. Sơ đồ khối tổng quát bộ thanh ghi

Chi tiết các chân tín hiệu của bộ thanh ghi được mô tả theo bảng sau:

Bảng 3.2. Danh sách các chân tín hiệu của bộ thanh ghi

Tên tín hiệu	Độ rộng (bit)	Loại tín hiệu	Mô tả
clk	1	Tín hiệu vào	Tín hiệu xung clock
rst_n	1	Tín hiệu vào	Tín hiệu reset, tích cực mức thấp
addr	12	Tín hiệu vào	Địa chỉ thanh ghi cần truy cập
wr_en	1	Tín hiệu vào	Tín hiệu cho phép ghi vào thanh ghi
rd_en	1	Tín hiệu vào	Tín hiệu cho phép đọc thanh ghi
pwdata	32	Tín hiệu vào	Dữ liệu ghi vào thanh ghi
prdata	32	Tín hiệu ra	Dữ liệu đọc từ thanh ghi
tx_data	16	Tín hiệu ra	Dữ liệu nạp vào thanh ghi TX FIFO
div_val	8	Tín hiệu ra	Giá trị chia tần số cho bộ chia xung

error	1	Tín hiệu ra	Cờ báo lỗi khi truy xuất thanh ghi không hợp lệ
cpol	1	Tín hiệu ra	Tín hiệu cấu hình bit CPOL cho khối SPI master
cpha	1	Tín hiệu ra	Tín hiệu cấu hình bit CPHA cho khối SPI master
wls	1	Tín hiệu ra	Cấu hình độ rộng dữ liệu truyền và nhận của SPI master
cdte	1	Tín hiệu ra	Cấu hình chế độ truyền liên tục cho SPI master
en_tx_empty	1	Tín hiệu ra	Cấu hình chế độ ngắt khi thanh ghi TX FIFO trống cho khối interrupt controller
en_tx_full	1	Tín hiệu ra	Cấu hình chế độ ngắt khi thanh ghi TX FIFO đầy cho khối interrupt controller
en_rx_empty	1	Tín hiệu ra	Cấu hình chế độ ngắt khi thanh ghi RX FIFO trống cho khối interrupt controller
en_rx_full	1	Tín hiệu ra	Cấu hình chế độ ngắt khi thanh ghi RX FIFO đầy cho khối interrupt controller
tx_wr_en	1	Tín hiệu ra	Cho phép ghi dữ liệu vào thanh ghi TX FIFO
rx_rd_en	1	Tín hiệu ra	Cho phép đọc dữ liệu từ thanh ghi RX FIFO
ss	2	Tín hiệu ra	Cấu hình chọn Slave cho SPI master
rx_data	16	Tín hiệu vào	Dữ liệu được đọc từ thanh ghi RX FIFO
tx_empty	1	Tín hiệu vào	Thông báo trạng thái trống của thanh ghi TX FIFO
tx_full	1	Tín hiệu vào	Thông báo trạng thái đầy của thanh ghi TX FIFO
rx_empty	1	Tín hiệu vào	Thông báo trạng thái trống của thanh

			ghi RX FIFO
rx_full	1	Tín hiệu vào	Thông báo trạng thái đầy của thanh ghi RX FIFO
busy	1	Tín hiệu vào	Thông báo trạng thái hoạt động của SPI master

Bảng 3.2 mô tả các tín hiệu của bộ thanh ghi trong hệ thống, bao gồm tín hiệu vào để điều khiển truy cập, ghi dữ liệu, trạng thái hoạt động, cũng như tín hiệu ra cung cấp dữ liệu đọc, cấu hình chế độ và chọn Slave. Các tín hiệu liên quan đến FIFO như tx_empty, tx_full, rx_empty, rx_full giúp giám sát tình trạng bộ đệm, trong khi tín hiệu busy phản ánh trạng thái bận của SPI master.

3.2.4.2 Hệ thống thanh ghi

Bộ thanh ghi chiếm dụng không gian địa chỉ 4KB từ 0x4000_2000 đến 0x4000_2018. Tất cả các vùng reserved trong vùng trên sẽ trả về 0 khi đọc (RAZ – Read-As-Zero) và bị bỏ qua khi ghi (WI – Write-Ignored). Chức năng của các vùng địa chỉ hợp lệ được thể hiện qua bảng sau:

Bảng 3.3. Tóm tắt hệ thống thanh ghi

Địa chỉ offset	Tên thanh ghi	Mô tả chức năng
0x4000_2000	Thanh ghi điều khiển đường truyền (LCR)	Cấu hình định dạng dữ liệu truyền.
0x4000_2004	Thanh ghi hệ số chia (DLR)	Thiết lập giá trị chia tần số để tạo clock cho SPI.
0x4000_2008	Thanh ghi cho phép ngắt (IER)	Cấu hình cho hệ thống điều khiển tín hiệu ngắt.
0x4000_200C	Thanh ghi trạng thái FIFO (FSR)	Cung cấp trạng thái của FIFO.
0x4000_2010	Thanh ghi đệm truyền (TBR)	Lưu trữ dữ liệu chuẩn bị truyền ra SPI.
0x4000_2014	Thanh ghi đệm nhận (RBR)	Chứa dữ liệu vừa nhận được từ SPI.

Bảng trên mô tả bản đồ thanh ghi (register map) của khối SPI, trong đó mỗi địa chỉ offset tương ứng với một thanh ghi đảm nhiệm một chức năng riêng trong hệ thống. Các thanh ghi này bao gồm nhóm cấu hình (thiết lập chế độ truyền và tần số), nhóm điều khiển ngắt, nhóm trạng thái phản ánh tình trạng FIFO, và nhóm dữ liệu dùng cho quá trình truyền và nhận SPI. Phần địa chỉ còn lại được dự trữ cho mở rộng trong tương lai, giúp thiết kế dễ nâng cấp mà không thay đổi cấu trúc hiện tại.

- Thanh ghi điều khiển đường truyền (LCR):

Thanh ghi điều khiển đường truyền hay Line Control Register (LCR) có địa chỉ offset tại 0x4000_2000.

Bảng 3.4. Mô tả thanh ghi điều khiển đường truyền - LCR

Bit	Tên	Loại	Giá trị mặc định	Mô tả
31:6	rsvd	-	-	Dự trữ (không sử dụng)
5:4	SS	R/W	2'b00	Cấu hình slave được chọn: • 2'b00: chọn Slave 0 • 2'b01: chọn Slave 1 • 2'b10: chọn Slave 2 • 2'b11: chọn Slave 3
3	CDTE	R/W	0	Bật/tắt chế độ truyền dữ liệu liên tục: • 0: Tắt (disable) • 1: Bật (enable)
2	CPHA	R/W	0	Cấu hình Clock Phase của SPI
1	CPOL	R/W	0	Cấu hình Clock Polarity của SPI
0	WLS	R/W	0	Cấu hình độ rộng khung dữ liệu: • 0: 8 bit • 1: 16 bit

Thanh ghi LCR được sử dụng để cấu hình và điều khiển hoạt động truyền nhận của SPI. Các bit trong thanh ghi cho phép lựa chọn độ dài dữ liệu từ (8-bit

hoặc 16-bit), thiết lập cực tính và pha xung clock (CPOL, CPHA), cũng như chế độ truyền liên tục (CDTE), chọn thiết bị Slave để giao tiếp (SS). Nếu chọn cùng lúc nhiều Slave để giao tiếp, hệ thống sẽ báo lỗi thông qua tín hiệu PSLVERR.

- Thanh ghi hệ số chia (DLR):

Thanh ghi hệ số chia hay Divisor Latches Register (DLR) có địa chỉ offset tại 0x4000_2004.

Bảng 3.5. Mô tả thanh ghi chốt hệ số chia - DLR

Bit	Tên	Loại	Giá trị mặc định	Mô tả
31:8	rsvd	-	-	Dự trữ (không sử dụng)
7:0	div_val	R/W	0	Hệ số chia tần số

Thanh ghi DL dùng để thiết lập hệ số chia tần số cho hoạt động tạo SCLK của SPI master. SCLK được tạo ra với công thức (3.1). Nhờ đó, SPI có thể điều chỉnh tốc độ truyền nhận dữ liệu linh hoạt theo yêu cầu hệ thống.

- Thanh ghi cho phép ngắt (IER):

Thanh ghi cho phép ngắt hay Interrupt Enable Register (IER) có địa chỉ offset tại 0x4000_2008.

Bảng 3.6. Mô tả thanh ghi cho phép ngắt - IER

Bit	Tên	Loại	Giá trị mặc định	Mô tả
31:5	rsvd	-	-	Dự trữ (không sử dụng)
3	en_rx_fifo_empty	R/W	0	Cho phép ngắt khi RX FIFO rỗng
2	en_rx_fifo_full	R/W	0	Cho phép ngắt khi RX FIFO đầy
1	en_tx_fifo_empty	R/W	0	Cho phép ngắt khi TX FIFO rỗng
0	en_tx_fifo_full	R/W	0	Cho phép ngắt khi TX FIFO đầy

Thanh ghi IER cho phép bật hoặc tắt các nguồn ngắt liên quan đến FIFO truyền (TX) và nhận (RX). Người dùng có thể cấu hình để kích hoạt ngắt khi FIFO rỗng hoặc đầy.

- Thanh ghi trạng thái FIFO (FSR):

Thanh ghi trạng thái FIFO hay FIFO Status Register (FSR) có địa chỉ offset tại 0x4000_200C.

Bảng 3.7. Mô tả thanh ghi trạng thái FIFO - FSR

Bit	Tên	Loại	Giá trị mặc định	Mô tả
31:5	rsvd	-	-	Dự trữ (không sử dụng)
3	rx_fifo_empty_status	RO	1	Trạng thái RX FIFO rỗng
2	rx_fifo_full_status	RO	0	Trạng thái RX FIFO đầy
1	tx_fifo_empty_status	RO	1	Trạng thái TX FIFO rỗng
0	tx_fifo_full_status	RO	0	Trạng thái TX FIFO đầy

Thanh ghi FSR cung cấp đầy đủ thông tin trạng thái hiện tại của 2 FIFO trong SPI. Các bit trạng thái cho biết RX FIFO/TX FIFO đang rỗng hay đầy. Thanh ghi chỉ cho phép đọc (Read-Only), đảm bảo phản ánh đúng tình trạng phần cứng.

- Thanh ghi đệm truyền (TBR):

Thanh ghi đệm truyền hay Transmitter Buffer Register (TBR) có địa chỉ offset tại 0x4000_2010.

Bảng 3.8. Mô tả thanh ghi đệm truyền - TBR

Bit	Tên	Loại	Giá trị mặc định	Mô tả
31:16	rsvd	-	-	Dự trữ (không sử dụng)
15:0	tx_data	WO	16'h0000	Dữ liệu cần gửi qua SPI

Thanh ghi TBR là bộ đệm ghi dữ liệu cần truyền ra SPI. Đây là thanh ghi chỉ ghi (Write-Only), khi đọc sẽ luôn trả về giá trị cố định 8'h00. Sau khi dữ liệu được ghi vào thanh ghi, nó sẽ được chuyển sang TX FIFO sau 1 chu kỳ clk.

- Thanh ghi đệm nhận (RBR):

Thanh ghi đệm nhận hay Receiver Buffer Register (RBR) có địa chỉ offset tại 0x4000_2014.

Bảng 3.9. Mô tả thanh ghi đệm nhận - RBR

Bit	Tên	Loại	Giá trị mặc định	Mô tả
31:16	rsvd	-	-	Dự trữ (không sử dụng)
15:0	rx_data	RO	16'hxxxx	Dữ liệu đã nhận được từ SPI

Thanh ghi RBR là bộ đệm chứa dữ liệu nhận được từ SPI. Đây là thanh ghi chỉ đọc (Read-Only), và chỉ được ghi dữ liệu từ SPI master.

3.2.4.3 Phản hồi lỗi

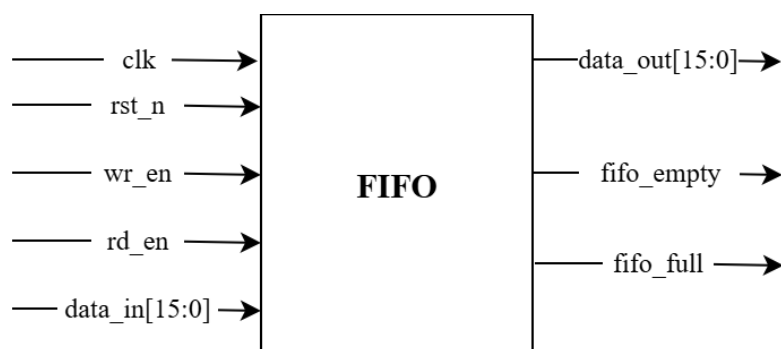
Tín hiệu error sẽ được kích hoạt và phản hồi cho APB slave khi Master thực hiện những giao dịch bị cấm như sau:

- Thay đổi các giá trị thiết lập trong thanh ghi LCR và DLR trong khi SPI master đang hoạt động.
- Truy cập vào ngoài vùng địa chỉ từ 0x4000_2000 đến 0x4000_2018.

3.2.5 Thiết kế khối TX FIFO và RX FIFO

3.2.5.1 Sơ đồ khối

Cả khối TX FIFO và RX FIFO được thiết kế trên 1 thanh ghi FIFO có sức chứa 16 word, mỗi word chứa 16 bit. Trong đó, TX FIFO có vai trò lưu những dữ liệu cần gửi thông qua chân MOSI và RX FIFO lưu trữ những dữ liệu đã nhận từ chân MISO của SPI master. Sơ đồ khối tổng quát của bộ FIFO được thể hiện như hình 3.4:



Hình 3.4. Sơ đồ khối tổng quát bộ FIFO

Chi tiết các chân tín hiệu của bộ FIFO được mô tả theo bảng sau:

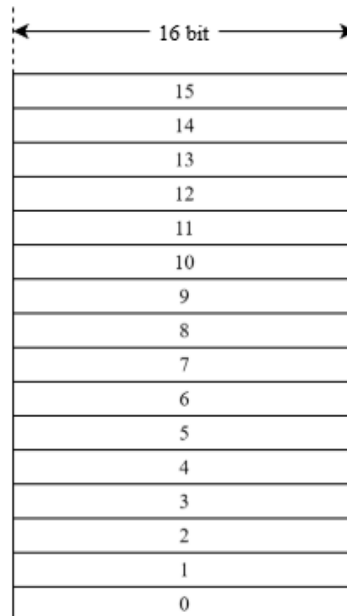
Bảng 3.10. Danh sách các chân tín hiệu của bộ FIFO

Tên tín hiệu	Độ rộng (bit)	Loại tín hiệu	Mô tả
clk	1	Tín hiệu vào	Tín hiệu xung clock
rst_n	1	Tín hiệu vào	Tín hiệu reset, tích cực mức thấp
wr_en	1	Tín hiệu vào	Tín hiệu cho phép ghi vào thanh ghi
rd_en	1	Tín hiệu vào	Tín hiệu cho phép đọc thanh ghi
data_in	16	Tín hiệu vào	Dữ liệu vào của thanh ghi
data_out	16	Tín hiệu ra	Dữ liệu đọc từ thanh ghi
fifo_empty	1	Tín hiệu ra	Cờ báo thanh ghi trống
fifo_full	1	Tín hiệu ra	Cờ báo thanh ghi đầy

Bảng 3.10 mô tả các tín hiệu chính của bộ FIFO 16-bit dùng cho cả TX FIFO và RX FIFO. Cụ thể, tín hiệu điều khiển bao gồm clk (xung clock), rst_n (reset mức thấp), wr_en (cho phép ghi) và rd_en (cho phép đọc). Dữ liệu được đưa vào qua data_in[15:0] và đọc ra tại data_out[15:0]. Hai cờ trạng thái fifo_empty và fifo_full lần lượt cho biết FIFO đang rỗng hay đã đầy, đảm bảo việc truy xuất dữ liệu không bị sai lệch. Ngoài ra, tín hiệu rst_n dùng để khởi tạo lại FIFO ở trạng thái ban đầu.

3.2.5.2 Cách hoạt động

Bộ thanh ghi FIFO trong hệ thống đóng vai trò như bộ đệm trung gian, giúp quản lý và cân bằng luồng dữ liệu giữa bus APB và giao tiếp SPI. Dữ liệu trong FIFO sẽ tuân theo thứ tự vào trước - ra trước. Mỗi FIFO có sức chứa 16 word, mỗi word 16 bit được thể hiện thông qua hình sau:



Hình 3.5. Cấu trúc bộ FIFO

Khi ghi dữ liệu, mỗi giá trị mới sẽ được lưu vào vị trí hiện tại của bộ đệm, sau đó con trỏ ghi được dịch chuyển sang vị trí kế tiếp. Việc ghi chỉ được cho phép khi FIFO chưa đầy, nhờ đó tránh ghi đè lên dữ liệu chưa được đọc.

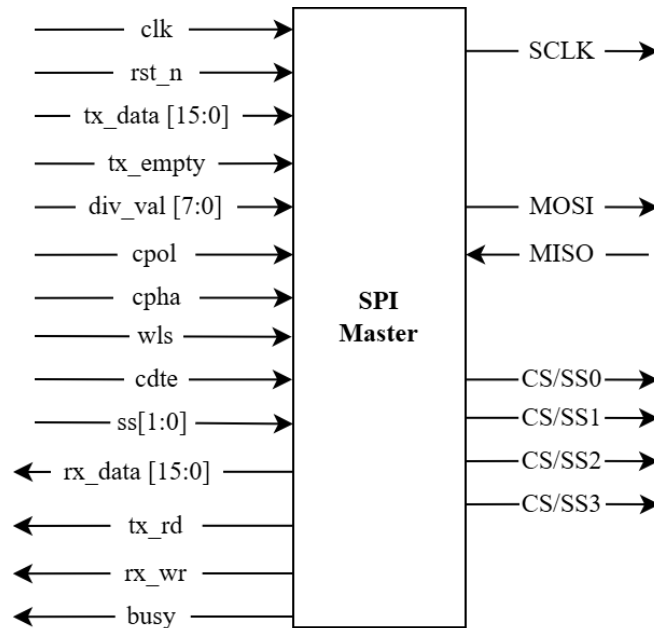
Khi đọc dữ liệu, hệ thống lấy dữ liệu tại vị trí mà con trỏ đọc đang trỏ tới, rồi con trỏ đọc cũng được tăng lên để trỏ sang phần tử tiếp theo. Quá trình đọc chỉ diễn ra khi FIFO không rỗng, đảm bảo dữ liệu đọc ra luôn hợp lệ.

Trạng thái FIFO được xác định dựa trên mối quan hệ giữa hai con trỏ đọc và ghi. FIFO được xem là rỗng khi hai con trỏ có cùng vị trí. FIFO được xem là đầy khi con trỏ ghi tiếp tục tăng và quay vòng đến vị trí của con trỏ đọc theo quy ước thiết kế. Cách xác định này cho phép hệ thống biết chính xác tình trạng FIFO trong quá trình hoạt động.

3.2.6 Thiết kế khối SPI Master

3.2.6.1 Sơ đồ khối

Khối SPI Master giữ vai trò trung tâm trong việc điều khiển quá trình truyền và nhận dữ liệu theo chuẩn SPI. Khối này hoạt động theo cấu hình được thiết lập từ bộ thanh ghi và thực hiện chính xác hành vi của một SPI master. Sơ đồ khối tổng quát của khối SPI Master được thể hiện như hình 3.6:



Hình 3.6. Sơ đồ khối tổng quát khối SPI Master

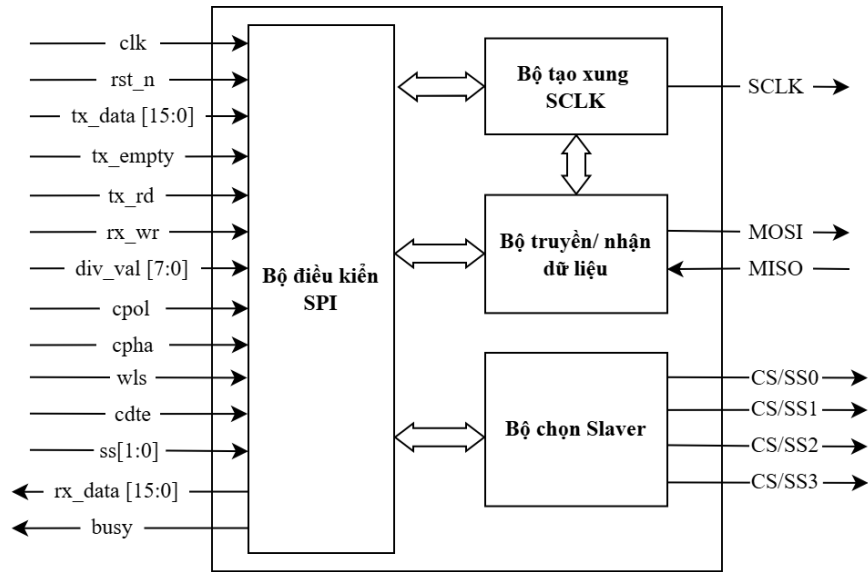
Chi tiết các chân tín hiệu của khối SPI Master được mô tả theo bảng sau:

Bảng 3.11. Danh sách chân tín hiệu của khối SPI Master

Tên tín hiệu	Độ rộng (bit)	Loại tín hiệu	Mô tả
clk	1	Tín hiệu vào	Tín hiệu xung clock
rst_n	1	Tín hiệu vào	Tín hiệu reset, tích cực mức thấp
tx_data	16	Tín hiệu vào	Dữ liệu cần gửi qua MOSI
tx_empty	1	Tín hiệu vào	Cờ báo thanh ghi TX FIFO
div_val	8	Tín hiệu vào	Hệ số chia cho bộ chia xung

cpol	1	Tín hiệu vào	Tín hiệu cấu hình CPOL
cpha	1	Tín hiệu vào	Tín hiệu cấu hình CPHA
wls	1	Tín hiệu vào	Tín hiệu cấu hình độ rộng khung dữ liệu truyền/ nhận
cdte	1	Tín hiệu vào	Tín hiệu cấu hình chế độ truyền liên tục/ không liên tục
ss	2	Tín hiệu vào	Tín hiệu chọn Slave đích
tx_rd	1	Tín hiệu ra	Tín hiệu cho phép đọc dữ liệu từ TX FIFO
rx_wr	1	Tín hiệu ra	Tín hiệu cho phép ghi dữ liệu vào RX FIFO
rx_data	16	Tín hiệu ra	Dữ liệu được đọc từ MISO
busy	1	Tín hiệu ra	Cờ thông báo SPI master đang trong quá trình truyền/ nhận dữ liệu
SCLK	1	Tín hiệu ra	Tín hiệu xung clock đầu ra
MOSI	1	Tín hiệu ra	Đường truyền đầu ra của SPI
MISO	1	Tín hiệu vào	Đường truyền đầu vào của SPI
CS/SS0	1	Tín hiệu ra	Tín hiệu chọn Slave 0, tích cực mức thấp
CS/SS1	1	Tín hiệu ra	Tín hiệu chọn Slave 1, tích cực mức thấp
CS/SS2	1	Tín hiệu ra	Tín hiệu chọn Slave 2, tích cực mức thấp
CS/SS3	1	Tín hiệu ra	Tín hiệu chọn Slave 3, tích cực mức thấp

Bên trong SPI master có các khối nhỏ đảm nhận những chức năng riêng được kết nối theo sơ đồ sau:



Hình 3.7. Sơ đồ kết nối các khối chính bên trong SPI Master

Khối SPI master bao gồm các khối chính sau:

Bộ tạo xung SCLK sử dụng tần số pclk của hệ thống kết hợp với hệ số chia được cấu hình ở thanh ghi DLR để tạo ra xung SCLK giúp đồng bộ với Slave trong quá trình truyền nhận dữ liệu.

Tần số của SCLK được tạo ra theo công thức:

$$f_{SCLK} \approx \frac{f_{PCLK}}{2 \times (div_val + 1)} \quad (3.1)$$

Trong đó:

- PCLK là tần số hệ thống được cấp cho IP thông qua bus APB.
- div_val là hệ số chia được cấu hình thông qua thanh ghi DLR trong bộ thanh ghi. Giá trị div_val là số nguyên dương từ 0-255
- SCLK là tần số xung được tạo ra qua PCLK.

Với mỗi mức tần số PCLK đầu vào, khoảng tần số SCLK tạo ra được sẽ dao động từ $\frac{PCLK}{512}$ đến $\frac{PCLK}{2}$. Ví dụ, với tần số đầu vào PCLK là 100MHz thì tần số SCLK có thể tạo ra từ khoảng 196KHz đến 50MHz.

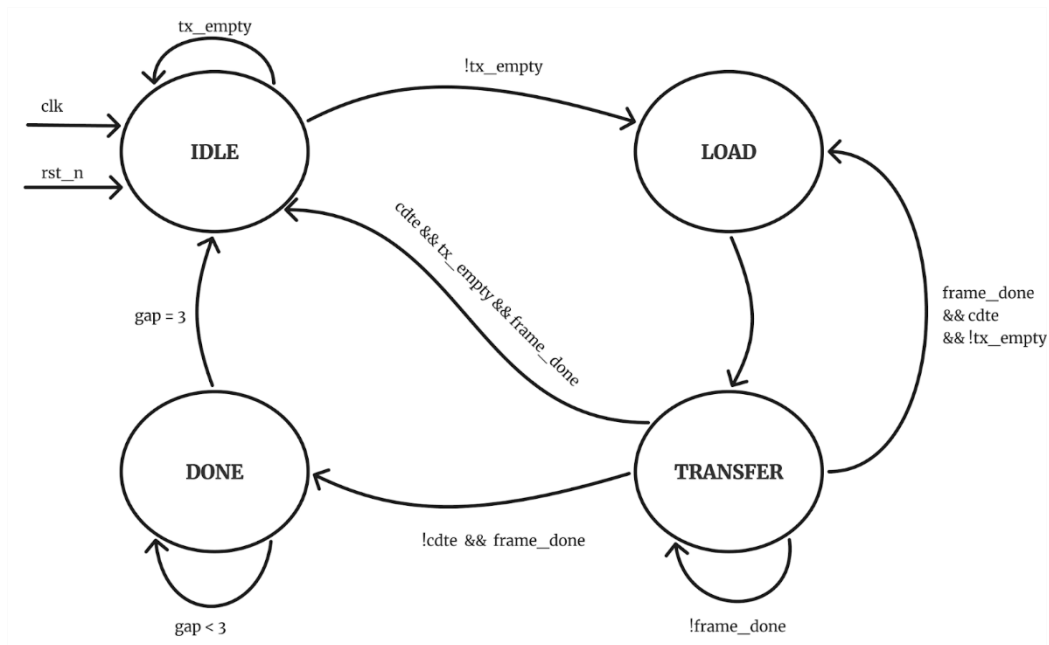
Bộ truyền/ nhận dữ liệu có nhiệm vụ dịch các bit cần gửi qua MOSI và lấy mẫu các bit dữ liệu được Slave truyền qua MISO. Khối hoạt động theo xung SPI ứng với các mode được cấu hình và quá trình hoạt động được kiểm soát bởi bộ điều khiển.

Bộ chọn Slaver là một bộ decoder chuyển các bit chọn Slave ban đầu sang các chân vật lý tương ứng.

Bộ điều khiển SPI là khối điều khiển trung tâm của SPI master, chịu trách nhiệm quản lý, điều khiển quá trình hoạt động của các khối trên trong 1 chu kỳ truyền đồng thời giao tiếp với FIFO trong việc lấy dữ liệu cần gửi và lưu dữ liệu đã nhận được.

3.2.6.2 Quy trình hoạt động

Bộ điều khiển SPI sẽ điều khiển khối SPI master hoạt động dựa trên mô hình máy trạng thái như sau:



Hình 3.8. Quy trình hoạt động bên trong khối SPI Master

Quy trình hoạt động của khối SPI master tuân theo 4 trạng thái của máy trạng thái hữu hạn FSM. FSM sẽ đổi trạng thái tại cạnh lên của clk và thỏa các điều kiện thích hợp của mỗi trạng thái.

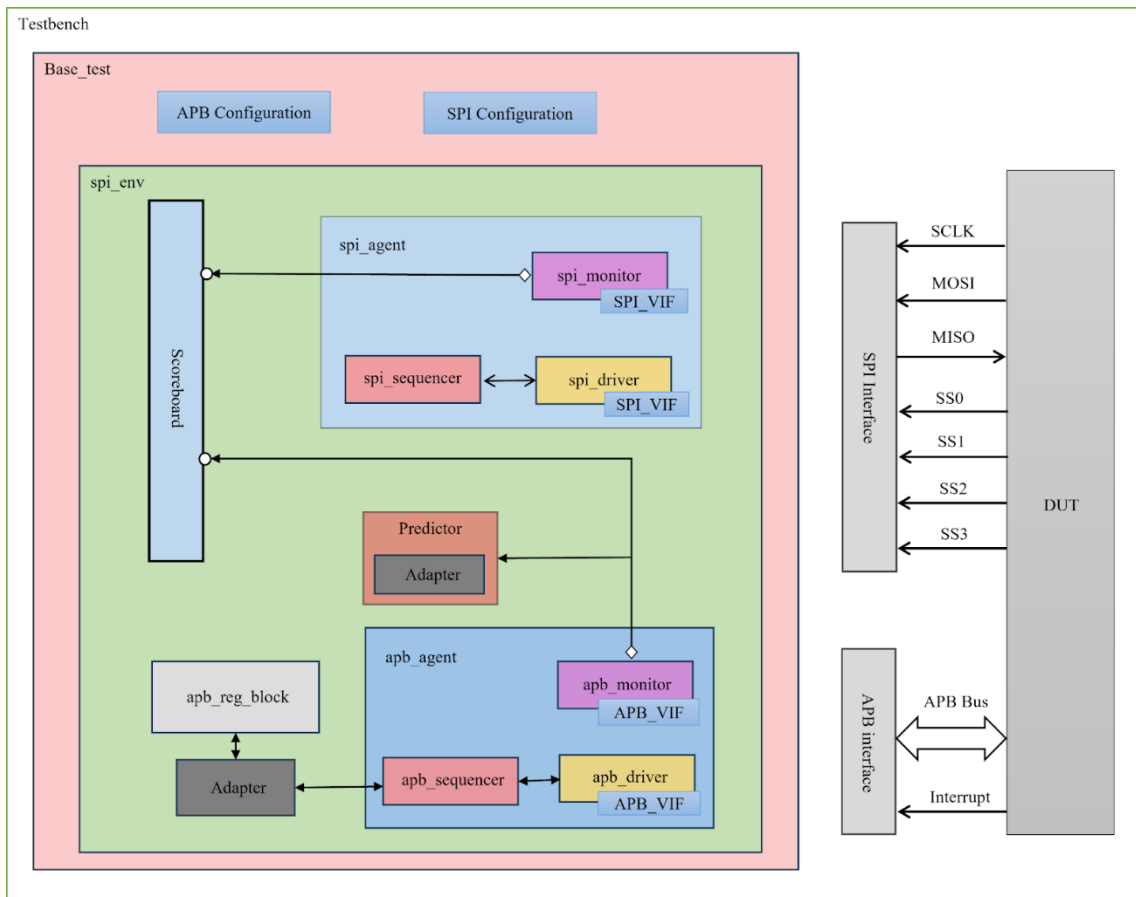
Trạng thái IDLE: Đây là trạng thái khởi đầu của khối SPI master. Ở trạng thái này, tín hiệu đầu ra *busy* sẽ ở mức thấp đồng thời chờ TX FIFO có dữ liệu. Khi TX FIFO có dữ liệu, *tx_empty* được hạ xuống mức thấp, trạng thái được chuyển sang LOAD.

Trạng thái LOAD: Ở trạng thái này, SPI master đọc dữ liệu từ TX FIFO bằng cách kích hoạt *tx_rd* và latch dữ liệu vào thanh ghi nội bộ chuẩn bị cho quá trình giao tiếp với Slave. Trong lúc đó, bộ chọn Slave sẽ được kích hoạt kéo chân SS tương ứng với tín hiệu *ss* đầu vào, đồng thời đặt đầu ra *busy* ở mức 1 thông báo SPI master bắt đầu hoạt động. Sau 1 clk, FSM sẽ chuyển sang trạng thái TRANSFER.

Trạng thái TRANSFER: Lúc này, bộ tạo xung SCLK được kích hoạt, đồng thời kích hoạt bộ truyền nhận dữ liệu. Bộ truyền dữ liệu tiến hành truyền dữ liệu đến Slave qua MOSI và lấy mẫu dữ liệu từ MISO tuân theo các chế độ đã cấu hình trước. Khi đã hoàn tất truyền khung dữ liệu với độ rộng đã cấu hình trước, cờ nội bộ *frame_done* được bật lên mức cao, *rx_wr* sẽ được bật lên để ghi dữ liệu đã lấy mẫu được vào RX FIFO. Nếu trong chế độ truyền liên tục với *cdte* = 1 và trong TX FIFO vẫn còn dữ liệu với *tx_empty* = 0, hệ thống sẽ chuyển về trạng thái LOAD để tiếp tục đọc dữ liệu tiếp theo từ FIFO để truyền dữ liệu mới. Nếu vẫn là trong chế độ truyền liên tục nhưng không còn dữ liệu trong TX FIFO, FSM sẽ chuyển trạng thái về IDLE. Với chế độ truyền không liên tục với *cdte* = 0, sau khi *frame_done* và ghi dữ liệu vào RX FIFO, FSM sẽ chuyển sang trạng thái DONE.

Trạng thái DONE: Ở đây, chờ 3 clk trước khi chuyển về trạng thái IDLE nhằm tạo khoảng nghỉ giữa hai lần truyền dữ liệu.

3.3 THIẾT KẾ MÔI TRƯỜNG KIỂM THỬ UVM



Hình 3.9. Cấu trúc môi trường kiểm thử UVM

Hình 3.9 minh họa cấu trúc môi trường kiểm thử UVM được sử dụng để kiểm chứng IP SPI giao tiếp qua APB. Bắt đầu với lớp Testbench, đây là lớp cao nhất đảm nhận việc giao tiếp trực tiếp với DUT qua các giao diện vật lý của APB và SPI (APB, SPI Interface). Các giao diện vật lý sẽ được testbench khởi tạo và ánh xạ sang các giao diện ảo (Virtual Interface - VIF) nhằm cung cấp cho các thành phần UVM dạng class sử dụng trong quá trình kiểm thử.

Dưới lớp `testbench` là lớp kiểm thử cơ bản `base_test`, nơi khởi tạo môi trường, thiết lập các tham số cấu hình và phân phối đến các thành phần cần thiết ở các lớp dưới kết hợp với các giao diện ảo đã nhận được từ `testbench`. Đồng thời khởi tạo và đạo diễn các kịch bản kiểm thử cho môi trường.

Khối *spi_env* là environment trung tâm của môi trường kiểm thử UVM, chịu trách nhiệm tổ chức và kết nối các thành phần kiểm thử chính của hệ thống, trong đó có 3 thành phần chính là *spi_agent*, *apb_agent* và *scoreboard*.

Lớp *spi_agent* đóng vai trò là SPI slave giao tiếp với giao diện SPI thông qua giao diện SPI ảo. Tại đây, *spi_sequencer* phối hợp với *spi_driver* điều phối các dữ liệu qua chân MISO đến DUT tương ứng với các kịch bản kiểm thử của *base_test*. Trong thời gian đó, *spi_monitor* ghi nhận toàn bộ tín hiệu xảy ra trên các chân SPI (SCLK, MOSI, MISO, SSx) và gửi các dữ liệu thu thập đến *scoreboard*.

Tương tự như *spi_agent*, *apb_agent* đóng vai trò giao tiếp với DUT thông qua giao diện APB ảo mà các lớp trên cung cấp trong việc cấu hình và điều khiển IP qua thao tác đọc/ghi thanh ghi của DUT qua giao thức APB. Thông tin của các thanh ghi được lưu tại *apb_reg_block* và được *adapter* chuyển thành các giao dịch đọc/ghi và gửi đến *apb_sequencer* và *apb_driver* sẽ dùng những thông tin đó để điều khiển các tín hiệu tương ứng với giao dịch đến DUT. Đồng thời, *apb_monitor* sẽ ghi lại toàn bộ các giao dịch APB và gửi đến *scoreboard* và *predictor*. *Predictor* sẽ phân tích các giao dịch cấu hình và điều khiển của IP để suy ra và tạo ra hành vi mong đợi của hệ thống.

Cuối cùng, *scoreboard* sẽ sử dụng tất cả những tham số cấu hình nhận được từ *base_test* kết hợp với những dữ liệu mà *spi_monitor* và *apb_monitor* ghi nhận được để tiến hành so sánh và đưa ra kết luận về tính đúng đắn trong hoạt động của IP.

3.4 THIẾT KẾ KỊCH BẢN KIỂM THỬ

Để kiểm tra tính đúng đắn và đầy đủ của IP SPI giao tiếp qua APB, các kịch bản kiểm thử được xây dựng nhằm bao phủ các chức năng cấu hình và hoạt động chính của hệ thống gồm 7 nhóm chính sau:

3.4.1 Kiểm tra hoạt động của các thanh ghi

Kịch bản tập trung vào việc sử dụng Register Abstraction Layer (RAL) nhằm đảm bảo việc truy cập thanh ghi đúng theo thiết kế ban đầu. Các thanh ghi sẽ

được kiểm tra các giá trị ban đầu, các thao tác đọc/ghi dữ liệu và kiểm tra phản hồi khi truy cập vào vùng dữ trữ. Từ đó thu được các giá trị ảnh xạ (mirror value) và so sánh với các giá trị mong muốn (desired value) được lưu trong reg model.

3.4.2 Kiểm tra tương thích với nhiều tần số APB

Ở nhóm này, DUT sẽ được hoạt động với các tần số thường thấy của APB bus đồng thời kiểm tra và đảm bảo hoạt động tạo tần số SCLK và việc truyền nhận dữ liệu của giao thức SPI đúng với thiết kế ban đầu. DUT sẽ được hoạt động trong các mức 20MHz, 50MHz, 100Mhz và một tần số ngẫu nhiên trong khoảng 20 - 100MHz.

3.4.3 Kiểm tra hoạt động SPI

Tại đây, DUT là được kiểm tra hoạt động của SPI thông qua việc kết hợp nhiều tổ hợp cấu hình. Các trường hợp kiểm tra kết hợp các cấu hình độ rộng dữ liệu truyền 8/16 bit, các chế độ SPI với CPOL và CPHA, giao tiếp một trong bốn Slave và kiểm tra trên cả chế độ truyền liên tục và không liên tục. Với mỗi cấu hình với tổ hợp tham số tương ứng và truyền 3 khung dữ liệu ở cả hai chiều, đảm bảo dữ liệu truyền nhận đều đúng và giao tiếp với đúng Slave.

3.4.4 Kiểm tra hoạt động của cấu hình ngắt

Phần này kiểm tra hoạt động của các trường hợp cấu hình ngắt bao gồm việc bật tắt các trường hợp ở thanh ghi IER. Song song với đó, kịch bản kiểm tra đảm bảo tín hiệu interrupt và trạng thái của thanh ghi FSR hoạt động đúng thiết kế ban đầu. Ngoài ra, nó còn giúp phát hiện các lỗi không đồng nhất giữa cấu hình của thanh ghi IER, tín hiệu đầu ra interrupt và trạng thái thanh ghi FSR.

3.4.5 Kiểm tra hoạt động cấu hình động

Mục này kiểm tra hoạt động tổng thể của DUT trong việc thay đổi cấu hình trong lúc hoạt động. Môi trường sẽ thiết lập DUT truyền nhận 3 khung dữ liệu liên tiếp với cấu hình đầu tiên. Sau khi thực hiện xong, môi trường thay đổi các tham số cấu hình hoạt động của DUT khác với ban đầu sau đó lại thực hiện lại việc truyền nhận trên và đảm bảo dữ liệu truyền và nhận đều đúng với kịch bản.

3.4.6 Kiểm tra hoạt động cấu hình lỗi

Đây là một mục không quá quan trọng nhưng cần thiết nhằm kiểm tra những trường hợp lỗi của IP. Lúc này, *base_test* sẽ cố tình cấu hình các tham số khác nhau cho DUT và môi trường như tham số độ rộng dữ liệu, tham số cấu hình các chế độ giao tiếp SPI, tham số chọn Slave và kiểm tra những phản hồi khi trong việc giao tiếp có lỗi đúng như kịch bản hay không.

3.4.7 Kiểm tra việc ghi đè cấu hình khi SPI đang hoạt động

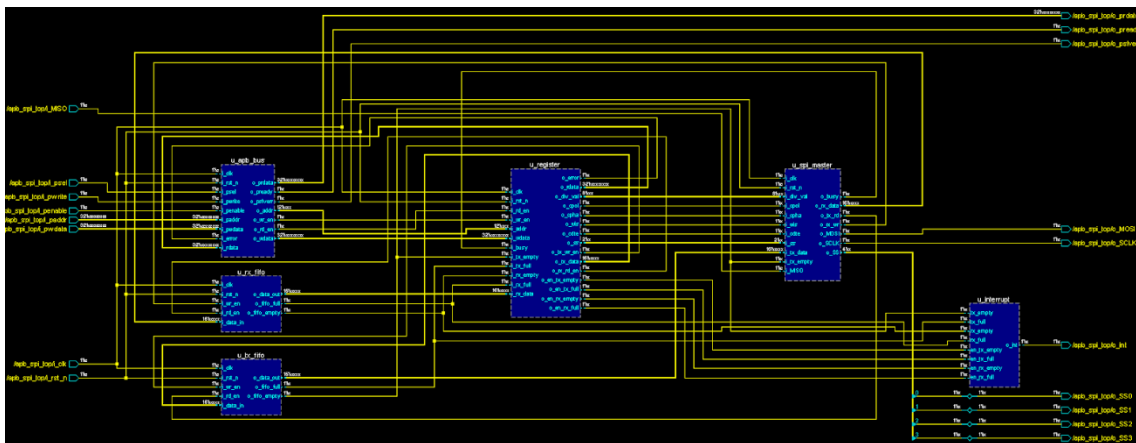
Mục kiểm thử này cố tình ghi đè các giá trị cấu hình đường truyền trong thanh ghi LCR và hệ số chia tần số trong thanh ghi DLR trong lúc SPI master đang trong giao dịch với Slave. Điều này nhằm kiểm tra quá trình phản hồi lỗi trong thiết kế ban đầu cũng như đảm bảo quá trình giao dịch vẫn diễn ra ổn định với cấu hình ban đầu và không ghi nhận cấu hình ghi đè.

CHƯƠNG 4

KẾT QUẢ

4.1 KẾT QUẢ THỰC HIỆN THIẾT KẾ PHẦN CỨNG

Sau quá trình phát triển và tích hợp trên phần mềm QuestaSim 2021.1, IP SPI giao tiếp với APB được hoàn thiện với các khối thành phần APB Bus, bộ thanh ghi (Register), các bộ nhớ FIFO và khối SPI master được kết nối với nhau như sơ đồ sau:



Hình 4.1. Sơ đồ nguyên lý IP SPI giao tiếp APB

Trong đó, các tín hiệu APB được xử lý trong khối APB bus ở bên trái, các tín hiệu của giao thức SPI được nối với khối SPI master ở bên trái đồng thời tín hiệu interrupt là đầu ra của khối Interrupt controller.

4.2 KẾT QUẢ THỰC HIỆN THIẾT KẾ MÔI TRƯỜNG KIỂM THỬ UVM

Song song với một IP Core đã hoàn thiện, môi trường kiểm thử UVM cũng đã được xây dựng thành công. Hình dưới đây trình bày kết quả cấu trúc phân cấp (Topology) được phản hồi ở cuối pha `end_of_elaboration`, ngay trước khi bắt đầu

mô phỏng, khẳng định sự kết nối chính xác và tính toàn vẹn của hệ thống trước giai đoạn mô phỏng.

```
# UVM_INFO /home/venus/Questasim/uvm-1.2/src/base/uvm_root.svh(579) @ 0: reporter [UVMTOP] UVM test
```

Name	Type	Size	Value
uvm_test_top	spi_non_8bit_cpol0_cpha0_slave0_test	-	@351
env	spi_environment	-	@376
apb_agt	apb_agent	-	@414
driver	apb_driver	-	@528
rsp_port	uvm_analysis_port	-	@547
seq_item_port	uvm_seq_item_pull_port	-	@537
monitor	apb_monitor	-	@557
apb_observe_port	uvm_analysis_port	-	@710
sequencer	apb_sequencer	-	@566
rsp_export	uvm_analysis_export	-	@575
seq_item_export	uvm_seq_item_pull_imp	-	@693
arbitration_queue	array	0	-
lock_queue	array	0	-
num_last_reqs	integral	32	'd1
num_last_rsps	integral	32	'd1
apb_predictor	uvm_reg_predictor #(apb_transaction)	-	@481
bus_in	uvm_analysis_imp	-	@490
reg_ap	uvm_analysis_port	-	@500
scoreboard	spi_scoreboard	-	@432
apb_export	uvm_analysis_imp_apb	-	@748
miso_export	uvm_analysis_imp_miso	-	@738
mosi_export	uvm_analysis_imp_mosi	-	@728
sclk_freq_export	uvm_analysis_imp_sclk_freq	-	@768
ss_export	uvm_analysis_imp_ss	-	@758
spi_agt	spi_agent	-	@423
driver	spi_driver	-	@921
rsp_port	uvm_analysis_port	-	@940
seq_item_port	uvm_seq_item_pull_port	-	@930
monitor	spi_monitor	-	@950
spi_observe_port_miso	uvm_analysis_port	-	@969
spi_observe_port_sclk_freq	uvm_analysis_port	-	@989
spi_observe_port_ss	uvm_analysis_port	-	@979
spi_observe_port_mosi	uvm_analysis_port	-	@959
sequencer	spi_sequencer	-	@784
rsp_export	uvm_analysis_export	-	@793
seq_item_export	uvm_seq_item_pull_imp	-	@911
arbitration_queue	array	0	-
lock_queue	array	0	-
num_last_reqs	integral	32	'd1
num_last_rsps	integral	32	'd1

Hình 4.2. Kết quả cấu trúc phân cấp

Dựa trên kết quả trích xuất của cấu trúc phân cấp (topology), môi trường kiểm thử đã chứng minh được sự đáng tin cậy trong quá trình mô phỏng. Điều đó được chứng minh qua sự đầy đủ của các thành phần trong hai tác nhân *apb_agt* đóng vai trò APB master và *spi_agt* có vai trò của SPI slave. Trong hai tác nhân trên, các thành phần *sequencer*, *driver*, *monitor* đều được tham gia hoạt động trong quá trình mô phỏng.

Hệ thống giám sát cũng được chứng minh hoạt động với các dữ liệu giám sát ở cả hai tác nhân đều được gửi về *scoreboard* thông qua cơ chế TLM. Trong đó, các thông tin giao dịch với APB bus được *apb_monitor* gửi từ *apb_obsever_port* đồng thời *spi_monitor* gửi lần lượt các thông tin liên quan đến tần số SCLK, dữ liệu truyền trên MOSI, MISO và chân Slave được chọn qua các cổng *spi_observe_port_sclk_freq*, *spi_observe_port_mosi*, *spi_observe_port_miso* và *spi_observe_port_ss*. Qua đó, *scoreboard* nhận các gói dữ liệu qua các cổng export tương ứng.

Kiến trúc kiểm thử được hoàn thiện bởi thành phần *apb_predictor*, đánh dấu sự tích hợp thành công của hệ thống thanh ghi (UVM RAL). Thành phần này đóng vai trò then chốt trong việc tự động dự đoán trạng thái thanh ghi và đồng bộ hóa dữ liệu với DUT.

4.3 HOẠT ĐỘNG CỦA HỆ THỐNG

4.3.1 Kiểm tra hệ thống thanh ghi

Trước khi thực hiện kiểm tra truyền nhận dữ liệu phức tạp, hệ thống cần đảm bảo khả năng giao tiếp chính xác giữa *apb_agent* đóng vai trò là APB master với DUT và hệ thống thanh ghi. Quá trình xác minh thanh ghi được tiến hành tự động hóa hoàn toàn thông qua UVM RAL. Các kịch bản kiểm thử tích hợp sẵn của UVM như *uvm_reg_hw_reset_seq* có đọc các giá trị mặc định và *uvm_reg_bit_bash_seq* cho việc ghi và đọc từng bit sử dụng đã khai báo. Tất cả được kích hoạt để rà soát toàn bộ không gian địa chỉ.


```

# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    89
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
#
# ** Report counts by id
# [NO_DPI_TSTNAME]    1
# [REG_PREDICT]       6
# [RNTST]             1
# [RegModel]          6
# [STARTING_SEQ]      1
# [TEST_DONE]         1
# [UVM/COMP/NAMECHECK]    1
# [UVM/RELNOTES]       1
# [UVM/REPORT/CATCHER]  1
# [UVMTOP]            1
# [apb_agent]         1
# [apb_monitor]       13
# [build_phase]        4
# [connect_phase]      2
# [final_phase]        2
# [reg_default_test]   4
# [run_phase]         18
# [spi_agent]          1
# [spi_reg2apb_adapter] 18
# [uvm_reg_hw_reset_seq] 6
#

```

Hình 4.3. Kết quả kiểm tra giá trị mặc định của các thanh ghi

Thông qua kịch bản kiểm tra tự động *uvm_reg_hw_reset_seq*, bộ dự đoán (Predictor) đã thực hiện thành công 6 chu kỳ so sánh – tương ứng với tổng số lượng thanh ghi được quy hoạch trong thiết kế phần cứng – mà không phát sinh bất kỳ cảnh báo lỗi nào (UVM_ERROR : 0). Kết quả này xác thực rằng tín hiệu Reset đã tác động hiệu quả lên toàn bộ khối chức năng, đảm bảo mọi giá trị thực tế đọc về từ phần cứng đều trùng khớp hoàn toàn với các tham số mặc định (Parameters) được quy định trong mã nguồn RTL.

Đối với quá trình đọc ghi của thanh ghi, kết quả được thể hiện như sau:

```

# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 6005
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
#
# ** Report counts by id
# [NO_DPI_TSTNAME] 1
# [REG_PREDICT] 576
# [RNTST] 1
# [RegModel] 576
# [STARTING_SEQ] 1
# [TEST_DONE] 1
# [UVM/COMP/NAMECHECK] 1
# [UVM/RELNOTES] 1
# [UVM/REPORT/CATCHER] 1
# [UVMTOP] 1
# [apb_agent] 1
# [apb_monitor] 1153
# [build_phase] 4
# [connect_phase] 2
# [final_phase] 2
# [reg_rw_test] 4
# [run_phase] 1800
# [spi_agent] 1
# [spi_reg2apb_adapter] 1728
# [uvm_reg_bit_bash_seq] 150
#

```

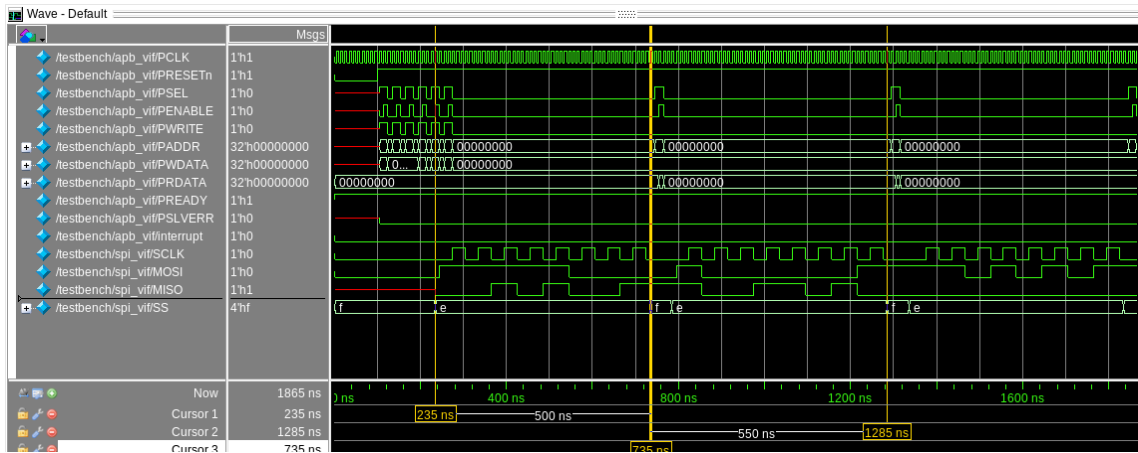
Hình 4.4. Kết quả kiểm tra giá trị trong quá trình đọc/ghi của các thanh ghi

Độ tin cậy của hệ thống thanh ghi được kiểm chứng nghiêm ngặt ở cấp độ bit thông qua kịch bản *uvm_reg_bit_bash_seq*. Số liệu thống kê 150 chuỗi kiểm tra xác nhận hệ thống đã thực hiện rà soát toàn diện khả năng lật trạng thái logic (0/1) của từng bit trong thanh ghi. Kết hợp với 576 lần đối chiếu thành công từ bộ dự đoán và kết quả UVM_ERROR : 0, có thể khẳng định hệ thống không chỉ chính xác trong logic điều khiển mà còn đảm bảo tính toàn vẹn dữ liệu tuyệt đối trên từng đường mạch của thanh ghi.

4.3.2 Kiểm tra hoạt động truyền nhận dữ liệu

4.3.2.1 Chế độ truyền không liên tục

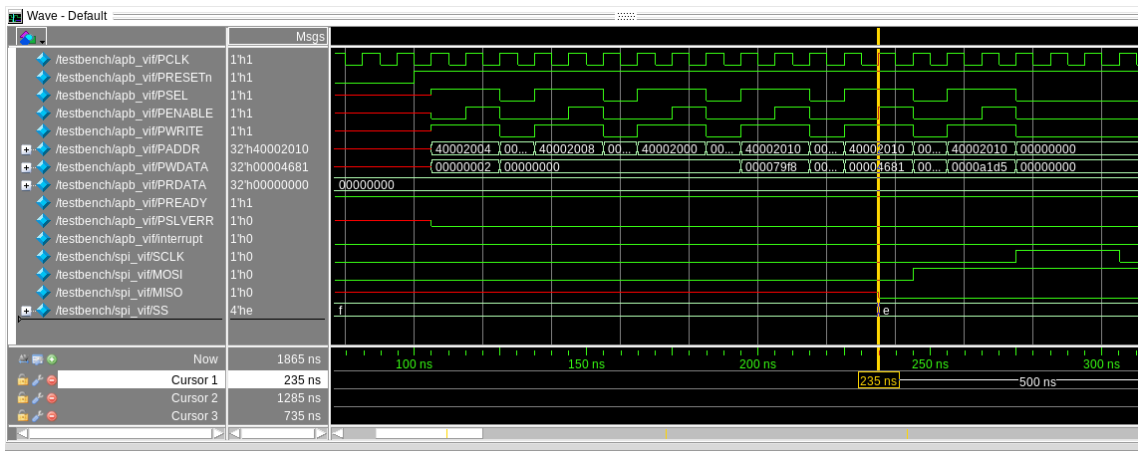
Kết quả quá trình truyền không liên tục với cấu hình ngẫu nhiên được thể hiện như hình sau:



Hình 4.5. Kết quả quá trình truyền không liên tục

Quá trình truyền nhận dữ liệu với chế độ không liên tục gồm 3 giai đoạn chính. Giai đoạn cấu hình từ đầu đến 235ns kết thúc lúc SS bắt đầu thay đổi, giai đoạn truyền dữ liệu từ sau giai đoạn cấu hình đến lúc 735ns và sau đó là đọc dữ liệu ra từ APB.

Giai đoạn cấu hình được thể hiện như sau:

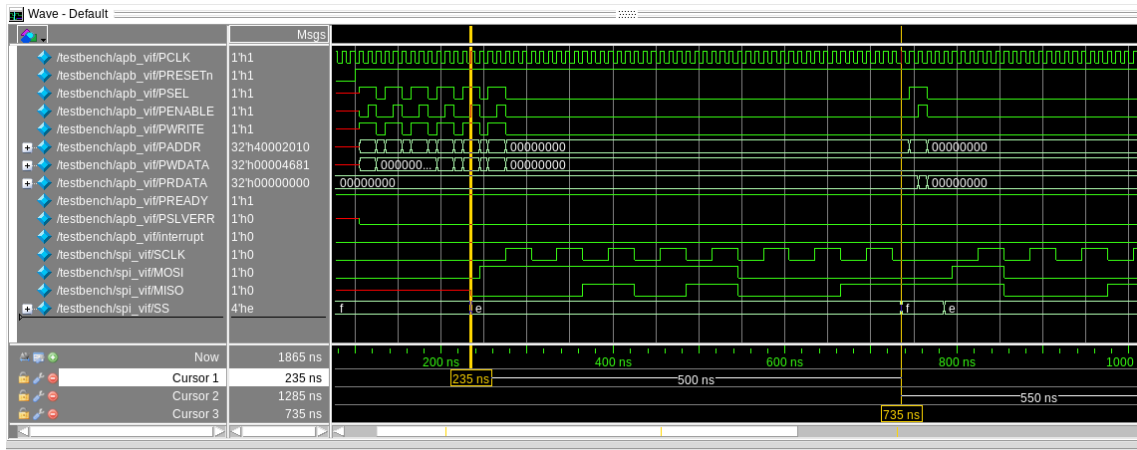


Hình 4.6. Giai đoạn cấu hình

Sau khi PRESETn được đặt lên mức cao, hệ thống APB bắt đầu ghi những cấu hình lần lượt lên các thanh ghi. Bắt đầu với thanh ghi 0x40002004 (DLR) với hệ số chia tần số là 2, thanh ghi 0x40002008 (IER) với dữ liệu 0 để tắt tín hiệu ngắt, thanh ghi 0x40002000 (LCR) với dữ liệu 0 với cấu hình độ rộng khung dữ liệu 8 bit, mode 0 với cpol = 0 và cpha = 0, chế độ không liên tục và chọn Slave 0. Tiếp theo, thanh ghi 0x40002010 (TBR) được ghi liên tiếp 3 dữ liệu ứng với 3

khung dữ liệu trong quá trình truyền đi. Ngay sau khi ghi dữ liệu đầu tiên vào TBR, tín hiệu SS của SPI lập tức chọn đúng Slave 0 để bắt đầu quá trình truyền nhận dữ liệu.

Quá trình truyền nhận dữ liệu được diễn ra như sau:

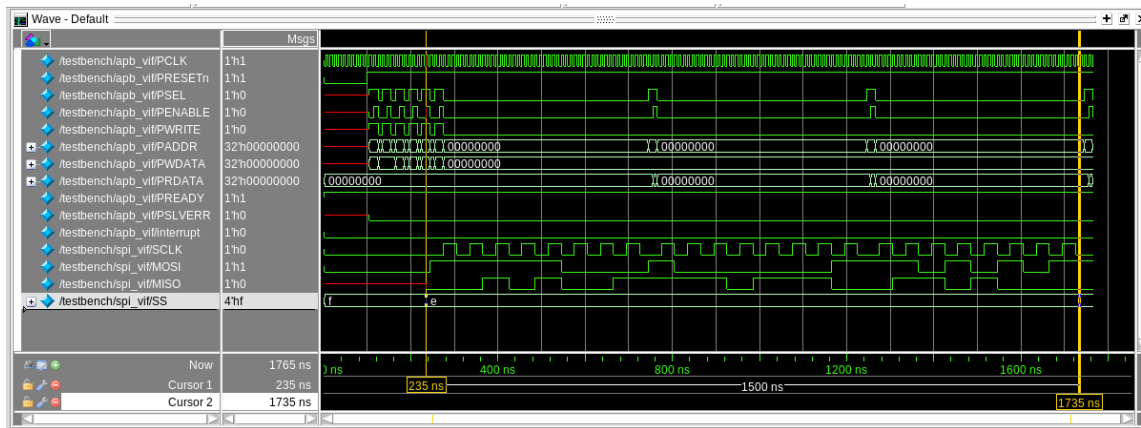


Hình 4.7. Quá trình truyền nhận dữ liệu

Sau khi SS được chọn, xung SCLK được tạo theo cấu hình CPOL và hoạt động truyền nhận được bắt đầu tuân theo cấu hình CPHA. Đến khi đủ khung dữ liệu đã cấu hình, SS được nhả về 1, môi trường tiến hành đọc kết quả qua thanh ghi 0x40002014 để kiểm tra tín hiệu DUT nhận được và so sánh. Đồng thời SS sẽ nghỉ 4 chu kỳ xung PCLK cho đến khi bắt đầu quá trình truyền nhận mới. Quá trình lặp lại cho đến khi hết 3 lần truyền nhận theo kịch bản.

4.3.2.2 Chế độ truyền liên tục

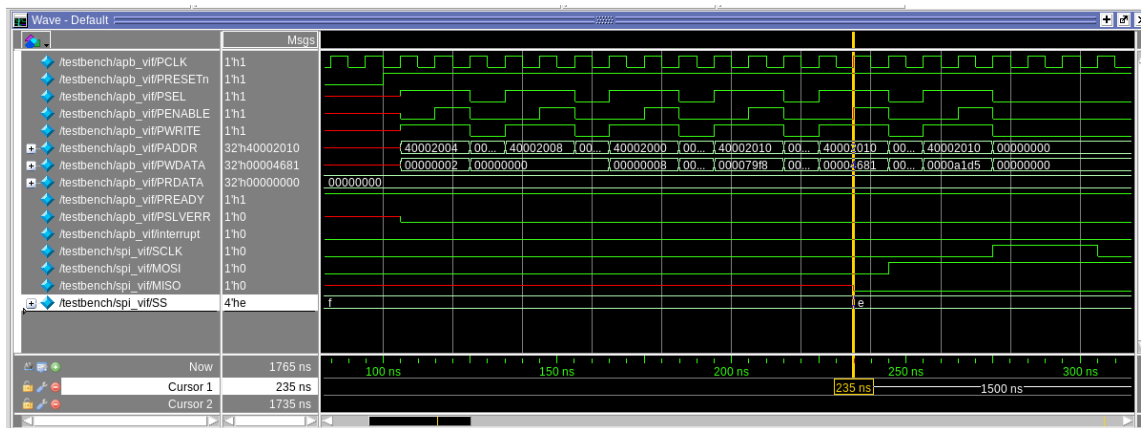
Ở đây, kịch bản sử dụng những cấu hình giống như quá trình truyền không liên tục, chỉ bật chế độ truyền liên tục ở bit thứ 3 thanh ghi LCR. Quá trình tổng quát được thể hiện như sau:



Hình 4.8. Kết quả quá trình truyền liên tục

Giống như chế độ không liên tục, quá trình truyền nhận cũng được chia là hai giai đoạn chính với thiết lập từ 0 - 235ns và giai đoạn truyền nhận dữ liệu là phần còn lại

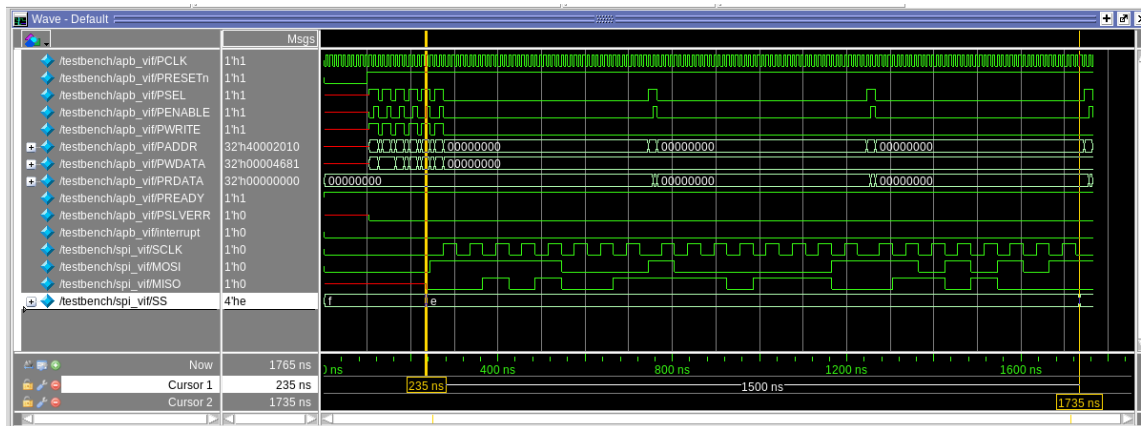
Giai đoạn cấu hình của chế độ truyền liên tục được thể hiện như sau:



Hình 4.9. Cấu hình của chế độ truyền liên tục

Các hoạt động cấu hình thanh ghi đều giống như chế độ không liên tục, chỉ khác ở đây thay vì ghi dữ liệu 32'h0 thành ghi 0x40002000 thì lúc này được ghi là 32'h8 và sau khi ghi dữ liệu đầu tiên vào 0x40002010 thì cũng là lúc SS được kéo xuống để bắt đầu giai đoạn truyền nhận.

Giai đoạn truyền nhận của chế độ truyền liên tục được thể hiện như sau:

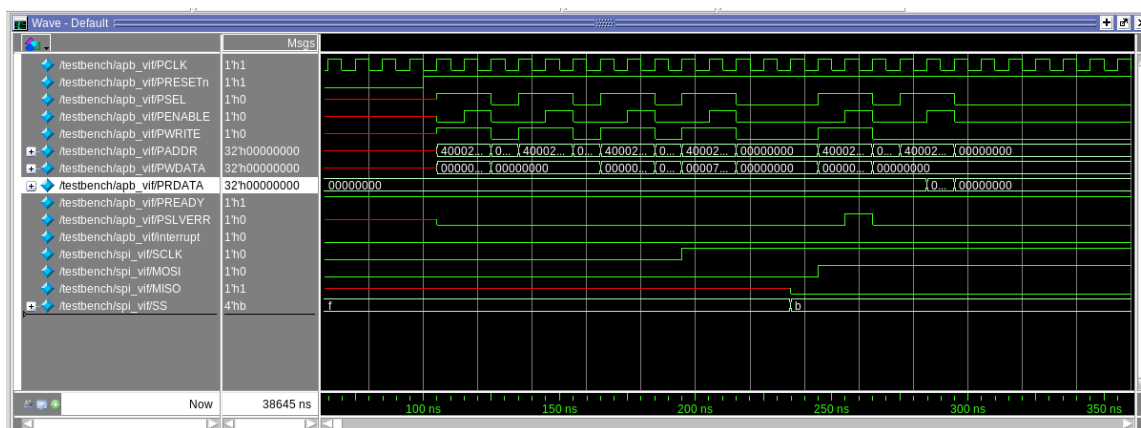


Hình 4.10. Truyền nhận của chế độ truyền liên tục

DUT kéo SS xuống cho đến khi truyền hết dữ liệu trong TBR. Giữa những frame dữ liệu, SCLK nghỉ 2 chu kỳ PCLK trước khi bắt đầu khung dữ liệu tiếp theo. Trong lúc đó, môi trường sẽ đọc dữ liệu trong RBR để đối chiếu các gói dữ liệu.

4.3.3 Kiểm tra hoạt động chống ghi đè LCR và DLR khi SPI đang hoạt động

Kịch bản kiểm tra chống ghi đè LCR được thể hiện như sau:

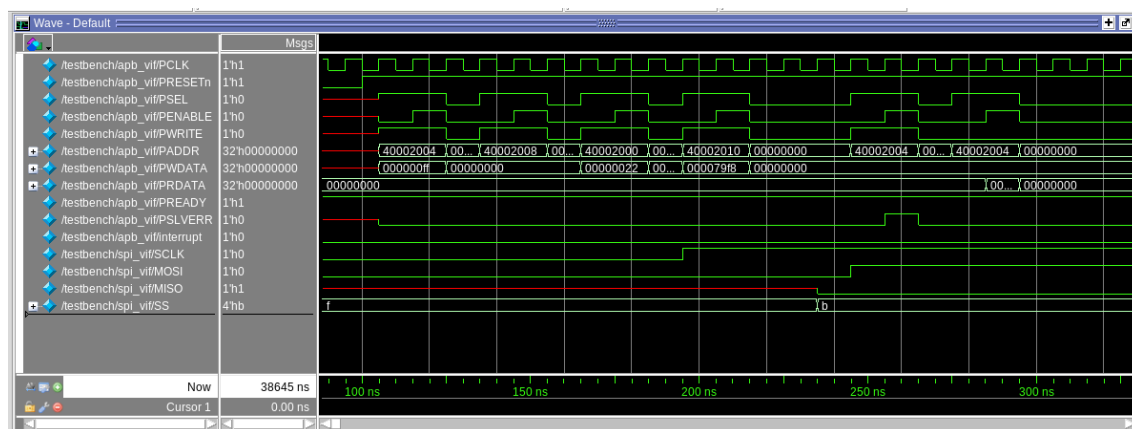


Hình 4.11. Kiểm tra chống ghi đè LCR

Sau khi reset, các thanh ghi được cấu hình đầy đủ như trong hoạt động kiểm tra truyền nhận dữ liệu. Sau khi SS được kéo xuống thông báo SPI master bắt đầu hoạt động, môi trường tiến hành ghi một dữ liệu khác vào 0x40002000 (LCR). Lúc này, ở giai đoạn truy cập của APB, PSLVERR được bật lên báo hoạt động

ghi bị lỗi và sau khi đọc lại thanh ghi LCR thì dữ liệu cho ra là giá trị cấu hình ban đầu cho đây đã ngăn thành công.

Kiểm tra chống ghi đè đối với thanh ghi DLR như hình sau:



Hình 4.12. Kiểm tra chống ghi đè DLR

Với kịch bản tương tự, DUT đã cho thấy khả năng chống ghi đè bằng các phản hồi với PSLVERR và không ghi giá trị trong giao dịch lỗi đó.

4.3.4 Đánh giá độ bao phủ chức năng

Mô hình đánh giá độ bao phủ đảm bảo các kịch bản kiểm tra đã bao quát đầy đủ các trường hợp hoạt động của thiết kế. Mục tiêu là định lượng mức độ hoàn thành của quá trình xác minh dựa trên các thông số giao dịch thực tế. Các điểm kiểm tra tập trung vào các cấu hình tần số của APB (PCLK), các cấu hình đường truyền của SPI như độ rộng khung dữ liệu (word), chế độ truyền nhận (cpol, cpha), chế độ truyền liên tục (continuous mode), các Slave được chọn (slave_id) được kiểm tra kết hợp các cấu hình với nhau.

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
/env_pkg/spi_scoreboard		100.00%				
TYPE SPI_COVERGROUP		100.00%	100	100.00%		✓
CVP SPI_COVERGROUP::PCLK		100.00%	100	100.00%		✓
bin PCLK[20]		58	1	100.00%		✓
bin PCLK[50]		60	1	100.00%		✓
bin PCLK[100]		54	1	100.00%		✓
CVP SPI_COVERGROUP::data_width		100.00%	100	100.00%		✓
bin data_width[8]		86	1	100.00%		✓
bin data_width[16]		80	1	100.00%		✓
CVP SPI_COVERGROUP::cpol		100.00%	100	100.00%		✓
bin cpol[0]		92	1	100.00%		✓
bin cpol[1]		80	1	100.00%		✓
CVP SPI_COVERGROUP::cpha		100.00%	100	100.00%		✓
bin cpha[0]		84	1	100.00%		✓
bin cpha[1]		88	1	100.00%		✓
CVP SPI_COVERGROUP::countinous_mode		100.00%	100	100.00%		✓
bin countinous_mode[0]		88	1	100.00%		✓
bin countinous_mode[1]		84	1	100.00%		✓
CVP SPI_COVERGROUP::slave_id		100.00%	100	100.00%		✓
bin slave_id[0]		40	1	100.00%		✓
bin slave_id[1]		44	1	100.00%		✓
bin slave_id[2]		40	1	100.00%		✓
bin slave_id[3]		48	1	100.00%		✓
CROSS SPI_COVERGROUP::word_cpol_cpha_cdtie_slave		100.00%	100	100.00%		✓

Hình 4.13. Mô hình đánh giá độ bao phủ chức năng

Quá trình cho thấy tất cả các chức năng đều được kết hợp và kiểm tra một cách toàn diện với độ bao phủ đạt 100% ở tất cả các mục. Chi tiết kịch bản kiểm tra và các hạng mục kiểm tra được thể hiện trong Vplan được đính kèm ở phần Phụ lục.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Trong quá trình nghiên cứu đề tài “Thiết kế IP SPI giao tiếp APB và kiểm thử với UVM”, nhóm đã thực hiện thiết kế và mô phỏng thành công một khối IP SPI hoạt động ở chế độ master, có khả năng giao tiếp với bus APB3 và đáp ứng đầy đủ các yêu cầu chức năng đã đặt ra ban đầu. IP được mô tả ở mức RTL bằng ngôn ngữ Verilog, đồng thời được kiểm thử và xác minh chức năng thông qua môi trường kiểm thử theo chuẩn UVM. Kết quả mô phỏng cho thấy IP SPI hoạt động ổn định, đúng chuẩn giao thức và phù hợp với các mục tiêu nghiên cứu của đề tài.

5.1.1 Kết quả đạt được so với mục tiêu đặt ra

So với mục tiêu ban đầu, khối IP SPI đã được thiết kế hoàn chỉnh ở mức RTL, đóng vai trò là cầu nối giữa bus APB3 và chuẩn truyền thông SPI. IP hỗ trợ đầy đủ các tín hiệu cơ bản của SPI, cho phép truyền và nhận dữ liệu theo đúng cơ chế của SPI master, đồng thời hỗ trợ cấu hình các tham số quan trọng như CPOL, CPHA, độ rộng dữ liệu 8 bit hoặc 16 bit và hệ số chia xung SCLK.

IP được thiết kế để hỗ trợ giao tiếp với tối đa bốn slave thông qua các tín hiệu chip select độc lập, đồng thời cho phép chế độ truyền liên tục. Cơ chế FIFO cho cả kênh truyền và kênh nhận được tích hợp nhằm giảm tải cho bus APB và tăng khả năng xử lý dữ liệu liên tục.

Bên cạnh việc thiết kế phần cứng, môi trường kiểm thử theo chuẩn UVM đã được nhóm xây dựng nhằm kiểm tra và xác minh chức năng của IP SPI. Thông qua mô phỏng, các kịch bản truyền nhận dữ liệu, thay đổi cấu hình và xử lý ngắt

đã được kiểm tra, cho thấy IP hoạt động đúng theo yêu cầu và không phát sinh lỗi chức năng trong phạm vi kiểm thử.

5.1.2 Ưu nhược điểm của hệ thống đã thiết kế

Ưu điểm: Hệ thống IP SPI được thiết kế có kiến trúc mang tính mô-đun rõ ràng giúp thuận tiện cho việc mở rộng, chỉnh sửa và tái sử dụng IP. Hệ thống tuân thủ nghiêm ngặt giao thức AMBA APB3 và chuẩn giao tiếp SPI nên giúp IP có thể dễ dàng tích hợp vào các hệ thống SoC. IP tích hợp FIFO và cơ chế ngắt giúp cải thiện hiệu suất và giảm sự phụ thuộc vào việc truy cập bus APB liên tục từ phía APB master. Việc sử dụng UVM trong xác minh chức năng làm tăng độ tin cậy và khả năng tái sử dụng cao cho thiết kế.

Nhược điểm: IP chỉ hỗ trợ chế độ SPI master và chưa hỗ trợ SPI slave, số lượng SPI slave giao tiếp còn hạn chế. Thiết kế mới dừng lại ở mức RTL và mô phỏng, chưa được kiểm chứng thông qua quá trình tổng hợp, phân tích thời gian (timing analysis) hay triển khai trên FPGA hoặc ASIC. Ngoài ra, IP hiện tại chỉ tương thích với chuẩn APB3 và chưa hỗ trợ các giao thức bus có hiệu năng cao hơn như AHB hay AXI.

5.2 HƯỚNG PHÁT TRIỂN

Trong tương lai, IP có thể được mở rộng để hỗ trợ thêm chế độ SPI slave, giúp tăng khả năng linh hoạt khi tích hợp vào các hệ thống phức tạp. Bên cạnh đó, việc hỗ trợ thêm các giao thức bus khác như AHB hoặc AXI sẽ giúp IP dễ dàng tích hợp vào các SoC thực tế.

Ngoài ra, IP có thể được tiếp tục phát triển sang các bước sau thiết kế RTL như tổng hợp, phân tích timing và triển khai trên FPGA hoặc ASIC nhằm đánh giá hiệu năng và mức tiêu thụ tài nguyên phần cứng. Về mặt kiểm thử, môi trường UVM có thể được mở rộng với nhiều kịch bản phức tạp hơn để nâng cao mức độ tin cậy của quá trình xác minh. Những hướng phát triển này sẽ góp phần hoàn thiện IP SPI và đưa sản phẩm tiến gần hơn tới khả năng ứng dụng trong các hệ thống thực tế.

PHỤ LỤC

Source code: <https://github.com/Venus-Lv5/APB3-to-SPI>

Verification plan (Vplan):

https://docs.google.com/spreadsheets/d/1kij3UA2H_rJlkfPDFnXCWY8VOEuxAZG67gsDkdm9mA8/edit?usp=sharing

TÀI LIỆU THAM KHẢO

- [1] D. Anjali, A. Radhika, “Design and Verification of APB to SPI Interface,” *International Journal of VLSI System Design and Communication Systems*, vol. 4, no. 10, pp. 994–998, Oct. 2016.
- [2] ARM Ltd., *AMBA® APB Protocol Specification*, ARM IHI 0024D, Issue D (APB5), Apr. 2021. [Online]. Available:
<https://developer.arm.com/documentation/ih0024/latest/>
- [3] nandland, *SPI Master in FPGA, Verilog Code Example*. (May. 10, 2019). Accessed: Oct. 15, 2025. [Online Video]. Available:
<https://www.youtube.com/watch?v=TR0Pw89EuGk>
- [4] nandland, *spi-master*, GitHub repository. [Online]. Available:
<https://github.com/nandland/spi-master>
- [5] Logic Madness, *AMBA APB Protocol*, LogicMadness.com. [Online]. Available:
<https://logicmadness.com/apb-protocol/>
- [6] Chip Verify, *UVM Tutoria*, ChipVerify.com. [Online]. Available:
<https://www.chipverify.com/uvm/uvm-tutorial>
- [7] Đức Lê. (2025, Jan. 18). *Bắt đầu tìm hiểu SystemVerilog và UVM cho Verification sao cho hiệu quả*, ICTC.edu.vn. [Online]. Available:
<https://ictc.edu.vn/bat-dau-tim-hieu-systemverilog-va-uvm-cho-verification-sao-cho-hieu-qua/>
- [8] *UVM Topics*, VerificationGuide.com. [Online]. Available:
<https://verificationguide.com/uvm-topics/>